



UNIVERSIDAD POLITÉCNICA SALESIANA
SEDE CUENCA
CARRERA DE COMPUTACIÓN

**DISEÑO Y DESARROLLO DE UNA APLICACIÓN MÓVIL INTELIGENTE PARA EL
DIAGNÓSTICO Y MANTENIMIENTO VEHICULAR MEDIANTE EL ANÁLISIS DE
CÓDIGOS OBD-II, CON SERVICIOS DE INTELIGENCIA ARTIFICIAL PARA LA
DETECCIÓN TEMPRANA DE FALLAS Y LA OPTIMIZACIÓN DEL
MANTENIMIENTO PREVENTIVO.**

Trabajo de titulación previo a la obtención del
título de Ingeniero en Ciencias de la Computación

AUTORES: FRANKLIN ANDRES GUAPISACA AREVALO
JUAN FRANCISCO QUIZHPI FAJARDO
TUTOR: ING. MARCELO ESTEBAN FLORES VÁZQUEZ, MST.

Cuenca - Ecuador
2026

CERTIFICADO DE RESPONSABILIDAD Y AUTORÍA DEL TRABAJO DE TITULACIÓN

Nosotros, Franklin Andres Guapisaca Arevalo con documento de identificación N° 0150943777 y Juan Francisco Quizhpi Fajardo con documento de identificación N°0106418916; manifestamos que:

Somos los autores y responsables del presente trabajo; y, autorizamos a que sin fines de lucro la Universidad Politécnica Salesiana pueda usar, difundir, reproducir o publicar de manera total o parcial el presente trabajo de titulación.

Cuenca, 2 de febrero del 2026

Atentamente,

Franklin Andres Guapisaca Arevalo
0150943777

Juan Francisco Quizhpi Fajardo
0106418916

CERTIFICADO DE CESIÓN DE DERECHOS DE AUTOR DEL TRABAJO DE TITULACIÓN A LA UNIVERSIDAD POLITÉCNICA SALESIANA

Nosotros, Franklin Andres Guapisaca Arevalo con documento de identificación N° 0150943777 y Juan Francisco Quizhpi Fajardo con documento de identificación N° 0106418916, expresamos nuestra voluntad y por medio del presente documento cedemos a la Universidad Politécnica Salesiana la titularidad sobre los derechos patrimoniales en virtud de que somos autores del Proyecto técnico: “Diseño y desarrollo de una aplicación móvil inteligente para el diagnóstico y mantenimiento vehicular mediante el análisis de códigos OBD-II, con servicios de inteligencia artificial para la detección temprana de fallas y la optimización del mantenimiento preventivo.”, el cual ha sido desarrollado para optar por el título de: Ingeniero en Ciencias de la Computación, en la Universidad Politécnica Salesiana, quedando la Universidad facultada para ejercer plenamente los derechos cedidos anteriormente.

En concordancia con lo manifestado, suscribimos este documento en el momento que hacemos la entrega del trabajo final en formato digital a la Biblioteca de la Universidad Politécnica Salesiana.

Cuenca, 2 de febrero del 2026

Atentamente,

Franklin Andres Guapisaca Arevalo

0150943777

Juan Francisco Quizhpi Fajardo

0106418916

CERTIFICADO DE DIRECCIÓN DEL TRABAJO DE TITULACIÓN

Yo, Marcelo Esteban Flores Vázquez con documento de identificación N° 0102408978, docente de la Universidad Politécnica Salesiana, declaro que bajo mi tutoría fue desarrollado el trabajo de titulación: DISEÑO Y DESARROLLO DE UNA APLICACIÓN MÓVIL INTELIGENTE PARA EL DIAGNÓSTICO Y MANTENIMIENTO VEHICULAR MEDIANTE EL ANÁLISIS DE CÓDIGOS OBD-II, CON SERVICIOS DE INTELIGENCIA ARTIFICIAL PARA LA DETECCIÓN TEMPRANA DE FALLAS Y LA OPTIMIZACIÓN DEL MANTENIMIENTO PREVENTIVO., realizado por Franklin Andres Guapisaca Arevalo con documento de identificación N° 0150943777 y por Juan Francisco Quizhpi Fajardo con documento de identificación N° 0106418916, obteniendo como resultado final el trabajo de titulación bajo la opción Proyecto técnico que cumple con todos los requisitos determinados por la Universidad Politécnica Salesiana.

Cuenca, 2 de febrero del 2026

Atentamente,

Ing. Marcelo Esteban Flores Vázquez, Mst.

0102408978

DEDICATORIA Y AGRADECIMIENTO

Yo, Franklin Andres Guapisaca Arevalo, dedico esta tesis a mis padres, Blanca Arevalo y Sixto Guapisaca, quienes han estado conmigo en cada etapa de mi vida. Sus consejos, llenos de cariño y sabiduría, me han guiado para superar cualquier obstáculo que se ha presentado en mi camino. Les agradezco de todo corazón por confiar siempre en mí y por darme la oportunidad de seguir una carrera universitaria, haciendo posible que finalmente termine este sueño y me convierta en un profesional. Gracias a su apoyo incondicional y por darme todos los recursos necesarios, he podido seguir adelante con confianza. Por eso, esta meta no es solo mía, sino que la comparto también con mis padres, quienes nunca dudaron de mi capacidad, por lo que a ellos les dedico este gran logro con todo mi cariño y gratitud.

Yo, Juan Francisco Quizhpi Fajardo, dedico esta tesis a mis padres, Johny Quizhpi y Patricia Fajardo, quienes han sido la base de lo que soy. Desde mi ingreso a la universidad y aun en los momentos más difíciles, entre tropiezos y lágrimas, siempre estuvieron animándome y apoyándome de manera incondicional. Cada consejo lleno de cariño y sabiduría me fortaleció y me impulsó a superarme, incluso cuando yo mismo no creía en mis capacidades; ellos me enseñaron que siempre puedo lograr aquello que me propongo.

Agradezco profundamente su esfuerzo y sacrificio, por haber dejado de lado sus propios gustos para brindarme educación y permitirme convertirme en un profesional. De igual manera, dedico este logro a mi hermano, quien desde pequeño me enseñó el valor de la perseverancia, la lucha constante y el no rendirse jamás; a pesar de ser menor que yo, desde su ejemplo me demostró que no existen sueños pequeños, sino metas que deben lucharse con esfuerzo y determinación. Asimismo, agradezco a mi abuelita por su apoyo incondicional, especialmente en los momentos relacionados con mi salud, por su cariño, sus consejos y su presencia constante. Agradezco a la vida por haberme regalado una familia tan valiosa, llena de valores, que ha sido fundamental en mi formación personal y profesional.

También dedico este trabajo a una persona muy especial que llegó recientemente a mi vida, mi princesa, quien con sus palabras, su cariño y su confianza me ha demostrado que mis capacidades, en las manos correctas, solo pueden crecer; su apoyo en esta última etapa ha sido invaluable. Finalmente, agradezco a Dios por colocarme en el lugar en el que estoy, por bendecirme con personas tan únicas y especiales, y a todos ellos dedico este logro que, aunque construido entre lágrimas y esfuerzo, atesoraré siempre, pues representa el camino para ayudar a mi familia y a las personas que amo.

Nosotros, Franklin Andrés Guapisaca Arévalo y Juan Francisco Quizhpi Fajardo, expresamos nuestro sincero agradecimiento a cada uno de los docentes de la carrera de Computación, quienes, a través de los conocimientos impartidos en cada clase, contribuyeron de manera fundamental a nuestra formación académica y profesional.

De igual manera, agradecemos a la Universidad Politécnica Salesiana por brindarnos la oportunidad de formar parte de esta casa salesiana, inculcándonos valores, principios y un ambiente formativo que nos ha permitido crecer tanto en lo académico como en lo personal, y que nos ha encaminado a convertirnos en los profesionales que soñamos ser.

Un profundo agradecimiento al Ing. Marcelo Esteban Flores Vázquez, tutor de nuestra tesis, quien desde el inicio nos brindó la apertura necesaria para escucharnos y apoyarnos de manera constante. Su experiencia, conocimientos y valiosos consejos fueron esenciales para guiarnos durante este proceso, así como su disposición para aclarar nuestras inquietudes y acompañarnos a lo largo de este proceso educativo.

Finalmente, agradecemos a San Juan Bosco por permitirnos formar parte de esta casa salesiana y por formarnos bajo el ideal de ser buenos cristianos y honrados ciudadanos.

Resumen

El proyecto de titulación tiene como objetivo el desarrollo de una aplicación móvil que detecte, lea y visualice los códigos de diagnóstico del vehículo (DTC). El fin del aplicativo móvil es facilitar el análisis del estado del vehículo y la interpretación de las fallas automotrices. Para ello, se ha implementado una solución que se basa en la tecnología OBD, con la integración de hardware y software en lo que es una arquitectura de fácil acceso.

La solución propuesta hace uso de un microcontrolador ESP32 conectado a un adaptador OBD-II del fabricante Freematics; esta integración permite la comunicación con la ECU del vehículo. El sistema está en la capacidad de leer, interpretar y eliminar los códigos DTC, los cuales serán enviados por medio del ESP32 a la base de datos de Firebase. Siendo así, la aplicación móvil se servirá de datos rápidamente para su análisis y la respectiva recomendación sobre el estado del vehículo.

Para el procesamiento y la gestión de la información, se han desarrollado servicios de backend en Flask, los cuales se han desplegado en la nube, eliminando la dependencia de una computadora externa para el despliegue de los servicios. El aplicativo móvil hace uso de estos servicios para brindar al usuario la información sobre el estado del vehículo en un lenguaje claro e intuitivo. Además, se ha integrado una API de inteligencia artificial (GPT), la cual brindará soporte para el análisis de los códigos DTC y las recomendaciones respectivas.

El proceso de eliminación de códigos DTC está a cargo de la aplicación móvil, la cual hace solicitudes al backend; estas llegan al ESP32, donde son validadas para que, posteriormente, sean ejecutadas y se lleve a cabo la eliminación dentro de la ECU. Este proyecto se orienta a conductores, estudiantes y técnicos del campo automotriz, ya que les proporciona una herramienta que orienta en el diagnóstico vehicular, así como en la interpretación de fallas para la correcta toma de decisiones en la prevención de problemas graves. Esta solución demuestra la viabilidad de la integración de software y hardware en el diagnóstico automotriz.

Palabras clave: OBD-II, Códigos de diagnóstico (DTC), Diagnóstico automotriz, Internet de las cosas (IoT), Aplicación móvil, Inteligencia artificial.

Abstract

The degree project aims to develop a mobile application that detects, reads, and displays vehicle diagnostic codes (DTCs). The purpose of the mobile application is to facilitate the analysis of the vehicle's condition and the interpretation of automotive faults. To this end, a solution based on OBD technology has been implemented, with the integration of hardware and software in an easily accessible architecture.

The proposed solution uses an ESP32 microcontroller connected to an OBD-II adapter from the manufacturer Freemartins; this integration allows communication with the vehicle's ECU. The system is capable of reading, interpreting, and deleting DTC codes, which will be sent via the ESP32 to the Firebase database. The mobile application will then quickly use the data for analysis and to provide recommendations on the vehicle's condition.

For information processing and management, backend services have been developed in Flask and deployed in the cloud, eliminating the need for an external computer to deploy the services. The mobile application uses these services to provide users with information about the status of the vehicle in clear and intuitive language. In addition, an artificial intelligence API (GPT) has been integrated, which will provide support for the analysis of DTC codes and the respective recommendations.

The process of deleting DTC codes is handled by the mobile application, which sends requests to the backend; these arrive at the ESP32, where they are validated so that they can then be executed and deleted within the ECU. This project is aimed at drivers, students, and technicians in the automotive field, as it provides them with a tool that guides them in vehicle diagnostics and fault interpretation for correct decision-making in the prevention of serious problems. This solution demonstrates the viability of software and hardware integration in automotive diagnostics.

Keys words: OBD-II, Diagnostic Trouble Codes (DTC), Automotive Diagnostics, Internet of Things (IoT), Mobile Application, Artificial Intelligence.

ÍNDICE DE CONTENIDO

1. Introducción	9
2. Problema	10
3. Objetivos Generales y Específicos.....	12
4. Revisión de la literatura o fundamentos teóricos.	13
4.1. Sistema OBD-II y códigos de diagnóstico de fallas (DTC)	13
4.2. Protocolos de comunicación.....	15
4.3. Microcontrolador ESP32	16
4.4. Arduino IDE.....	16
4.5. Desarrollo de servicios web con Flask	16
4.6. Base de datos NoSQL con Firestore.....	17
4.7. Inteligencia artificial generativa y modelos de lenguaje (LLM)	17
4.8. Desarrollo de aplicaciones móviles con Flutter.....	17
4.9. Despliegue en la nube	18
5. Marco metodológico	19
5.1. Desarrollo	19
5.2 Despliegue.....	24
5.3. Testing	26
5.4. Desarrollo con metodología en espiral.....	27
6. Resultados	33
6.1 Resultados del sistema en un Chevrolet Spark Life	33
6.2 Resultados del sistema en una Great Wall Wingle.....	42
7. Cronograma	46
8. Presupuesto	50
9. Conclusiones.....	51
10. Recomendaciones.....	52
11. Uso de IA Generativa (GenAI).....	53
12. Referencias bibliográficas.....	53
13. Anexos.....	54

1. **Introducción**

Actualmente, con el aumento del número de vehículos y la dependencia de ellos para el transporte, ha incrementado la necesidad de garantizar condiciones óptimas de funcionamiento, seguridad y eficiencia. Sin embargo, los propietarios optan por realizar el mantenimiento cuando la falla es visible o ha comprometido de forma negativa el funcionamiento del vehículo. De esta manera, los propietarios son víctimas de elevados costos de reparación y la degradación de la vida útil de componentes del vehículo, incrementando así el riesgo de accidentes y un impacto ambiental por emisiones contaminantes.

Frente a esta problemática, los conocidos sistemas de diagnóstico a bordo o OBD-II se han colocado como una de las herramientas clave dentro de la industria automotriz. Con estos sistemas se logra supervisar y detectar fallas mediante los códigos de diagnóstico (DTC), los cuales dan conocimiento sobre anomalías en subsistemas como el motor, transmisión, sistema eléctrico y emisiones. Sin embargo, la interpretación de todos los datos requiere muchas de las veces el conocimiento técnico, una limitante para muchos de los propietarios de los vehículos.

Bajo este contexto, el avance de la inteligencia artificial y el desarrollo de aplicaciones móviles han abierto nuevas oportunidades para mejorar la forma en que los propietarios interactúan con la información del vehículo. Con la integración de tecnologías de (IoT), servicios en la nube y modelos de inteligencia artificial, se logra recolectar datos en tiempo real que serán analizados, interpretados y presentados de manera comprensible, facilitando la toma de decisiones orientadas al mantenimiento vehicular.

El presente trabajo de titulación aborda el diseño y desarrollo de una aplicación móvil inteligente orientada al diagnóstico y mantenimiento vehicular, su base es la interpretación de códigos (DTC) y la aplicación de herramientas de inteligencia artificial. Se presenta una herramienta accesible, intuitiva y eficiente que dé al usuario la información necesaria para comprender la naturaleza de la falla, su gravedad y el conocimiento para afrontarla sin requerir de conocimiento especializado.

2. Problema

Antecedentes

Al momento de hablar sobre el mantenimiento vehicular, como usuarios de automóviles, solemos hacerlo cuando existe una falla evidente; de esta manera, ignoramos las recomendaciones dadas por el fabricante. Esta realidad nos expone a varios problemas, ya sea por los altos costes de mantenimiento o por los gastos relacionados con las piezas de recambio que se ven afectadas por la falta de detección temprana de fallas, lo que puede reducir la vida útil del vehículo. Como resultado, es común encontrar automóviles en malas condiciones, con fallas que van desde problemas mecánicos en el motor hasta defectos en el sistema eléctrico, la suspensión y otras piezas esenciales.

Todas estas condiciones de riesgo no son solo para el conductor, sino para todas las personas de su entorno, ya que aumentan la posibilidad de accidentes, aumenta la contaminación del medio ambiente debido al desgaste del vehículo y crea situaciones peligrosas. Además, estas condiciones pueden generar sanciones legales para el propietario y disminuir el valor comercial del vehículo. Mantener un vehículo en buen estado es una responsabilidad individual y conlleva un compromiso no solo con la seguridad colectiva, sino también con el medio ambiente. Un vehículo en óptimas condiciones reduce los riesgos legales y técnicos, al tiempo que contribuye al bienestar público y a la protección del medio ambiente (Arrobo, 2021).

En la actualidad, el mantenimiento de los vehículos es un debate entre los dueños de automóviles, ya que la cultura a la que están acostumbrados los usuarios mantiene un enfoque reactivo; es decir, solo se toman medidas cuando ya se aprecia un fallo o el vehículo ha dejado de funcionar. Esta cultura contradice las sugerencias realizadas por expertos y fabricantes del sector automovilístico, lo que tiene implicaciones para la economía, la seguridad y el medio ambiente. La fiabilidad de los vehículos se ve limitada por el mantenimiento reactivo, ya que los fallos solo se diagnostican después de que se producen. Por otro lado, los métodos predictivos y preventivos favorecen una gestión más eficaz del estado de los vehículos, lo que mejora la seguridad y optimiza los costes (Kolhar y Mahale, 2025).

Los sistemas de diagnóstico como el OBD-II permiten obtener información relevante sobre el estado del vehículo gracias a la lectura de los códigos de diagnóstico de fallas (DTC). Estos códigos, generados por la unidad de control electrónico (ECU), permiten identificar

distintas fallas detectadas y constituyen una fuente importante de información, ya que representan una herramienta fácil de usar para los propietarios de vehículos (Michailidis, Panagiotopoulou y Papadakis, 2025).

En este escenario es crucial que exista una herramienta que permita extraer y mostrar los códigos DTC del vehículo y con uso de la inteligencia artificial, simplificar su interpretación de una manera fácil para todos los usuarios. Así, esta herramienta puede ofrecer sugerencias y orientación general basadas en estos códigos, lo que ayuda a tomar decisiones informadas sobre el mantenimiento del vehículo y promueve prácticas de mantenimiento preventivo, sin sustituir el diagnóstico profesional especializado.

Importancia y alcances

Para evitar que estos problemas nos afecten, se han analizado diversos métodos y herramientas que permiten la detección temprana de fallas. Esto brinda a los usuarios la oportunidad no solo de identificar los problemas, sino también de adquirir los conocimientos necesarios para abordarlos de manera adecuada. Por esta razón, es esencial que el estudio incluya los conceptos más relevantes de los documentos de referencia y que los métodos implementados estén correctamente interrelacionados. Esto garantiza que la información sea clara, coherente y precisa.

Con base en este análisis, se propone el desarrollo de un sistema capaz de acceder y mostrar los códigos de diagnóstico de fallas (DTC) extraídos de la computadora del vehículo. Este sistema se implementará mediante una aplicación móvil con una interfaz intuitiva y fácil de usar, que permitirá la conexión a través del adaptador Freematics OBD-II para facilitar la identificación de posibles problemas detectados.

La aplicación contará con el apoyo de inteligencia artificial, que ofrecerá interpretaciones asistidas de los códigos DTC, explicando su gravedad y diferenciando entre fallas críticas y menores. De esta manera, incluso los usuarios sin conocimientos técnicos podrán comprender las causas y las acciones recomendadas, ya sea realizar verificaciones básicas o acudir a un profesional, lo que les permitirá tomar decisiones rápidas y seguras.

3. Objetivos Generales y Específicos.

Objetivo General

Diseñar y desarrollar una aplicación móvil inteligente para el diagnóstico y mantenimiento vehicular, mediante el análisis de códigos OBD-II y la integración de inteligencia artificial, con el propósito de detectar tempranamente fallas y optimizar el mantenimiento preventivo.

Objetivos Específicos

- **OE1:** Analizar y sistematizar la información técnica relacionada con los protocolos de comunicación OBD-II empleados por diferentes marcas y modelos de vehículos, mediante la revisión de documentación técnica y bibliografía especializada, con el fin de establecer los requerimientos funcionales del sistema.
- **OE2:** Implementar un sistema embebido que establezca comunicación con la unidad de control electrónico (ECU) del vehículo a través de la interfaz OBD-II, encargado de recopilar, procesar y transmitir los datos de diagnóstico hacia la aplicación móvil, garantizando la correcta interpretación de los parámetros automotrices.
- **OE3:** Implementar la conexión con herramientas de inteligencia artificial para el análisis de los datos obtenidos del sistema OBD-II, con el fin de identificar patrones, predecir fallas y generar recomendaciones de mantenimiento preventivo.
- **OE4:** Desarrollar una aplicación móvil para Android que reciba, procese y visualice en tiempo real la información obtenida del sistema embebido, mediante una interfaz gráfica intuitiva y el uso de herramientas modernas de desarrollo móvil, facilitando el monitoreo y el diagnóstico de fallas vehiculares.
- **OE5:** Validar el funcionamiento y el desempeño del sistema propuesto mediante pruebas experimentales en diferentes vehículos, evaluando la precisión de los diagnósticos, la eficiencia de la comunicación y la efectividad de los servicios de inteligencia artificial utilizados.

4. Revisión de la literatura o fundamentos teóricos.

4.1. Sistema OBD-II y códigos de diagnóstico de fallas (DTC)

El ya conocido sistema de diagnóstico a bordo (OBD) surge a mediados de los años 70 y comienzos de los años 80, debido a que durante este tiempo, en Estados Unidos, se empezó la implementación de componentes electrónicos en los vehículos, los cuales formaban parte importante para el diagnóstico y control de parámetros del vehículo. Estos componentes

permitían detectar también errores o fallos que podían desencadenar daños tanto en el sistema eléctrico como en el mecánico del vehículo.

Al principio, los primeros parámetros que se controlaban eran los de gases contaminantes, especialmente considerando que en esa época comenzaron a endurecerse las normativas que regulan las emisiones de estos gases. A partir de eso, la finalidad del sistema fue orientándose hacia la prolongación de la vida útil de los vehículos y la reducción del número de averías. Es así como el OBD se desarrolló rápidamente, trayendo consigo varias funcionalidades que dieron lugar, en el año 1991, al nacimiento del estándar OBD I, que pasó a ser de uso obligatorio en todos los vehículos de ese año. Este estándar daba prioridad a la monitorización de las emisiones; sin embargo, era considerado poco eficiente, ya que apenas podía controlar otros parámetros. Esto llevó a que, en el año 1996, apareciera el estándar OBD-II, el cual ha prevalecido hasta la fecha.

Según Blasco, V. (2014) “OBD-II se trata de un sistema de diagnóstico integrado en la gestión del motor, ABS, etc. del vehículo, por lo tanto, es un programa instalado en las unidades de mando del motor”. Esta definición nos explica que el OBD-II actúa como un software embebido encargado de la supervisión de los sistemas críticos del vehículo. Sin embargo, el OBD-II es más conocido como un estándar global que, por medio de un conector universal y sus respectivos protocolos de comunicación, permite acceder a todos los datos que contiene la unidad de control del motor, más conocida como ECU, así como a otros módulos importantes.

La función principal del OBD-II se centra en el monitoreo constante de aspectos como el rendimiento del motor, los sistemas de control de emisiones y el almacenamiento de los códigos de fallas, más conocidos como DTCs, los cuales aparecen cuando se detecta una anomalía en el vehículo.

De esta manera, las capacidades del OBD-II lo convierten en una herramienta fundamental para el diagnóstico de fallas en vehículos, el mantenimiento eficiente de los mismos y el correcto cumplimiento de las normativas ambientales.

Para comprender cómo actúa OBD-II y su implicación en el diagnóstico de fallas, es importante conocer sobre los (DTC). Como señala Villamar Aguirre, I. D. (2008). “Para todos los sistemas OBD, si se encuentra un problema, la computadora enciende la luz “CHECK ENGINE” para advertir al conductor, y establece un Código de Diagnóstico de Problema (DTC) para identificar dónde ocurrió el problema”. Si abordamos más a fondo

lo que nos ofrecen los DTC, encontramos que estos se encuentran estandarizados con base en la normativa SAE J2012, la cual define una estructura de código compuesta por cinco caracteres (por ejemplo: P0123).

Si analizamos la estructura de estos códigos, tenemos que la primera letra es un indicador tanto de P (Powertrain), que está directamente relacionado con motor, transmisión y emisiones. C (Chassis), que abarca los sistemas del chasis del vehículo y el sistema de ABS. B (Body), que toma en cuenta aspectos tanto de carrocería, airbags, etc. U (Network), que se relaciona a los fallos presentes en la comunicación entre módulos, o más conocido como la red CAN.

No obstante, los códigos DTC también cuentan con dígitos que se encargan de identificar el subsistema y la falla que se presentó. Un ejemplo de esto es el código P0302, que nos dice que la falla se encuentra en P(motor), 0(código SAE), 3(sistema de encendido), 02 (falla en cilindro 2). Con este tipo de códigos, se proporciona al técnico la información necesaria sobre la falla, lo que permite llevar a cabo un diagnóstico y una corrección oportunos.

4.2. Protocolos de comunicación

Los protocolos según Lopes, J. C. O. (2014):

Es un conjunto de reglas y normas establecidas que permiten una comunicación exitosa entre dos o más dispositivos para el intercambio de información. Sí, se quiere acceder al ECU para diagnosticar el automóvil, se hace indispensable una herramienta de diagnóstico con el protocolo de comunicación que resida dentro del automóvil.

Dentro de lo que abarca los protocolos de comunicación, estos son clasificados en dos categorías tanto para protocolos lentos, en donde su base son estándares anteriores y también protocolos rápidos, los cuales son los estándares que se usan actualmente.

Al principio, los protocolos lentos coexistieron, como el J1850 (PWM/VPW) y el ISO 9141-2 (K-Line), en los que se presentaban velocidades limitadas y donde las variaciones dependen del fabricante del vehículo. Las limitaciones que estaban presentes en estos protocolos fueron mejorando dado que la evolución tecnológica llevó al surgimiento del protocolo CAN Bus (ISO 15765-4), el cual pasó a ser el estándar obligatorio.

Gracias a este protocolo hubo un salto inmenso tanto en aspectos como velocidad, robustez y la capacidad de interacción con el ECU, así como con la red de módulos que contienen datos tanto de ABS, transmisión, carrocería, etc. Este avance fue clave, ya que con él se materializó el concepto de un sistema de diagnóstico de fallas para los vehículos.

4.3. Microcontrolador ESP32

“El microcontrolador ESP32 es un SoC (System on a Chip) desarrollado por la empresa Espressif el cual se considera uno de los mejores microcontroladores del mercado para su uso en IoT debido a sus capacidades de conexión tanto de Bluetooth como de Wifi” Madariaga Revuelta, A. (2022). Este microcontrolador cuenta con un procesador de 32 bits que trabaja hasta en frecuencias de 240 MHz, además incorpora diversos pines tanto de entrada como de salida digital (GPIO), convertidores analógico-digitales, interfaces de comunicación UART, etc. Gracias a estas características, se alza como una alternativa versátil para el desarrollo de sistemas embebidos.

4.4. Arduino IDE

“Se trata de una aplicación multiplataforma la cual está disponible para los sistemas operativos Windows, Linux y macOS que se utiliza para realizar el diseño del código y poder cargar este código en la placa Arduino, tras ser compilado, pero también podemos usarlo con placas de otros proveedores” Barrios Portales, D. (2022). La filosofía que se emplea con esta aplicación multiplataforma es la de presentar en un lenguaje simplificado y una toolchain transparente, lo que le da la facilidad a todo tipo de usuarios, sin importar su experiencia, de programar microcontroladores.

Este IDE ha ido evolucionando y ha trascendido hasta convertirse en el entorno de desarrollo predilecto para los microcontroladores. Esto debido a que se encuentra potenciado por librerías de código abierto, permitiendo así la integración de hardware complejo y el desarrollo de aplicaciones de IoT, robótica y automatización.

4.5. Desarrollo de servicios web con Flask

“El framework Flask es considerado por muchos uno de los más ligeros y sencillos de utilizar. Algunos programadores lo consideran un “microframework” porque proporciona solo las características esenciales para desarrollar aplicaciones web” (Valverde González et al., 2025). Flask se define por su filosofía de diseño, priorizando la flexibilidad y el control que tiene el desarrollador. Con Flask se consigue un impacto directo en la

metodología de desarrollo, así como el fácil diseño y prototipado de API RESTful. De esta manera, Flask se convierte en la forma perfecta para la creación de endpoints que microcontroladores como el ESP32 pueden recibir y procesar.

4.6. Base de datos NoSQL con Firestore

Cloud Firestore funciona como una base de datos del tipo NoSQL que está alojada en la nube, ofreciendo escalabilidad y flexibilidad, la cual se encuentra respaldada por la infraestructura de Google Cloud. Permite la sincronización de datos y el almacenamiento en múltiples plataformas, con funcionalidades que aseguran el rendimiento en condiciones de red inestables o sin conexión, lo que la hace adecuada para soluciones modernas de alto rendimiento. Esta herramienta será de gran utilidad, ya que cumple su función en la nube al brindarnos la posibilidad de almacenar todos los códigos provenientes del auto a través del adaptador, como los códigos de diagnóstico DTC, facilitando su análisis, seguimiento y gestión desde cualquier dispositivo conectado.

4.7. Inteligencia artificial generativa y modelos de lenguaje (LLM)

El modelo de inteligencia artificial conocido como ChatGPT fue creada por la empresa de OpenAI, la cual hace uso de métodos avanzados de análisis del lenguaje natural, también ha sido entrenada con extensos volúmenes de texto, permitiendo entender el lenguaje humano y otorgar respuestas de manera coherente y contextualizada. ChatGPT está basado en el modelo de GPT-5, este es uno de los sistemas más avanzados hasta la fecha, gracias a esto, es capaz de hacer tareas como resumir textos, traducir, ofrecer información, generar imágenes y ejecutar otras funciones con gran precisión (Morales-Chan, 2023).

La IA de ChatGPT nos facilitará el análisis y la interpretación de los códigos de error provenientes de la ECU del vehículo. Funcionará como el cerebro que entiende los datos y los interpreta de modo que el usuario pueda comprender de forma sencilla el problema generado. De esta manera, ChatGPT podrá sugerir posibles soluciones, verificar y clasificar los códigos según su severidad (leve, moderada, grave), creando así diagnósticos y ayudas sobre el estado de su vehículo.

4.8. Desarrollo de aplicaciones móviles con Flutter

El framework de Flutter fue desarrollado por Google y utiliza el lenguaje de programación conocido como Dart, posee una gran ventaja, la cual es desarrollar aplicaciones tanto como

para las plataformas de Android, iOS, equipos de escritorio y para la web, y todo esto partiendo de una misma base de código. Flutter posee una interfaz moderna que está inspirada en Material Design y Cupertino, además de incorporar elementos visuales flexibles y fáciles de personalizar. También tiene una función llamada Hot Reload, dicha función permite realizar cambios en el código y verlos en tiempo real, agilizando así el proceso de desarrollo (Collaguazo, Venegas, Guerrero, Freire & Beltrán, 2022).

Con la ayuda de Flutter se podrá desarrollar la interfaz principal de la aplicación Android, será la principal interacción que tendrán los usuarios con el sistema de diagnóstico, en dicha aplicación se mostrará los datos provenientes del vehículo en tiempo real mediante paneles interactivos y fáciles de usar. Además, el usuario tendrá la posibilidad de consultar a la IA de ChatGPT, a través de la cual podrá recibir explicaciones y recomendaciones sobre los errores detectados. Flutter será de gran utilidad, ya que representa el punto de conexión entre las tecnologías del OBD-II, la nube y la IA, brindando la información técnica en datos útiles y comprensibles desde el teléfono móvil.

4.9. Despliegue en la nube

El despliegue en la nube permite que servicios, aplicaciones y todo tipo de infraestructura trabajen directamente sobre plataformas de computación en la nube, aprovechando su escalabilidad, disponibilidad y flexibilidad, gracias a este enfoque, se evita la necesidad de configurar e instalar software y hardware en servidores locales. El despliegue se realiza utilizando recursos de proveedores externos como Google Cloud, AWS y otras plataformas con funcionamiento similar, lo que permite aprovechar la infraestructura escalable y gestionada de forma remota. Así se logra reducir costos y tiempo de implementación, además de facilitar el mantenimiento, mejorar la seguridad y adaptarse mejor a los cambios tecnológicos que puedan surgir en el futuro. (Hernandez & Fuentes, 2014).

El backend del sistema estará alojado en la nube, específicamente en la plataforma Render, donde se encuentran las APIs encargadas de recibir los datos del ESP32, almacenarlos en la base de datos y establecer las conexiones con los servicios de inteligencia artificial. Al estar desplegado en la nube, el backend será accesible desde cualquier lugar, lo que garantiza que los datos y las consultas realizadas estén siempre disponibles. De esta manera, la aplicación móvil podrá acceder en todo momento a las versiones más recientes del servicio, asegurando la disponibilidad de la información vehicular procesada.

5. Marco metodológico

Este proyecto de titulación implementó la metodología en espiral. “El modelo espiral en el desarrollo del software es un modelo meta del ciclo de vida del software, donde el esfuerzo del desarrollo es iterativo, tan pronto culmina un esfuerzo del desarrollo, por ahí mismo comienza otro” (Galo Fariño, R., 2011). Los ciclos de vida, también llamados espirales, incluyeron etapas como la planificación, el análisis de riesgos, el desarrollo y la validación, etapas que permitieron generar versiones del proyecto que poco a poco evolucionaron hasta alcanzar su etapa final.

Para este proyecto, esta metodología permitió desarrollar e integrar cada uno de sus componentes, empezando desde la adquisición de los datos por parte del adaptador de Freematics y el módulo ESP32, hasta el servicio de Flask, la base de datos, la API de ChatGPT y la aplicación móvil Flutter. Bajo este enfoque, en cada iteración se produjo un módulo, el cual fue evaluado y mejorado antes de avanzar entre fases, logrando así un control sobre el proyecto y garantizando la evolución del sistema de diagnóstico vehicular.

5.1. Desarrollo

El desarrollo de nuestro aplicativo móvil partió de la investigación y documentación de la tecnología OBD presente en los vehículos automotores. Comprender qué es OBD, cómo este se integra con cada vehículo y dónde lo encontramos fue vital, ya que a partir de eso se comprendió el enfoque que tomaría el aplicativo móvil. Partimos con la investigación de cada uno de los protocolos de comunicación existentes en la industria automotriz. Se recolectaron los siguientes datos:

Características	SAE J1850 PWM	SAE J1850 VPW	ISO 9141-2	ISO 14230 KWP2000	ISO 15765 CAN
Norma	SAE	SAE	ISO	ISO	ISO
Velocidad Típica	41.6 kbps	10.4 kbps	10.4 kbps	10.4 kbps hasta 125 kbps	250 kbps hasta 500 kbps
Fabricantes	Ford Mazda Jaguar Volvo	General Motors Chrysler Toyota	Fabricantes europeos 1990-2000 Chrysler Toyota Hyundai, Kia	Fabricantes europeos 1990-2000 Kia, Hyundai Fabricantes japoneses	Todos los vehículos desde 2008 en Estados Unidos Para vehículos de gasolina en

					Europa 2001 y diésel 2004
--	--	--	--	--	---------------------------

Con estos datos, nuestro conocimiento se direccionó a la investigación de marcas y modelos comunes que cuentan con el protocolo OBD-II. En la siguiente tabla se presentan la marca, modelo y protocolo OBD-II de los vehículos con más presencia en Ecuador:

Marca	Modelos Comunes	Protocolo OBD II
Toyota	Corolla Hilux RAV4 Yaris	ISO 9141-2 / KWP2000 (1998–2007) ISO 15765 CAN (2008)
Chevrolet	Aveo / Sail Spark Cruze Optra	SAE J1850 VPW (1996–2006) ISO 15765 CAN (2006-2008)
Hyundai / Kia	Accent Rio Tucson Elantra	ISO 14230 / KWP2000 (2000s) ISO 15765 CAN (2007-2009)
Ford	Fiesta Focus Ranger Ecosport	SAE J1850 PWM (1996–2004) ISO 15765 CAN (2004–2008)
Renault	Logan Sandero Clio	ISO 9141 / KWP2000 (2000s) ISO 15765 CAN (2006-2009)
Mazda	3 6 BT-50	SAE J1850 PWM (Algunos modelos americanos) ISO 15765 CAN (2005–2008)
Suzuki	Swift Vitara	ISO 9141 / KWP2000 (2000s) ISO 15765 CAN (actualidad)

Con el conocimiento de los protocolos OBD-II y su presencia en los vehículos, el desarrollo requirió conocer los PIDs, los cuales son los identificadores que el módulo OBD-II requiere para solicitar datos a la ECU del vehículo. A continuación, se presenta algunos de ellos:

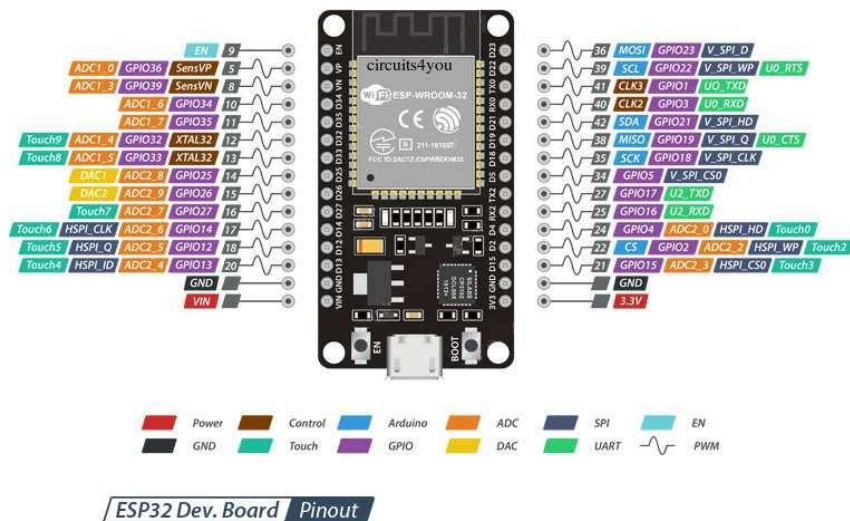
PID (hex)	Parámetro	Descripción	Unidad
03	Códigos de falla	Códigos de	Códigos

	almacenados (DTCs)	diagnóstico presentes	DTC
01	Estado del motor del sistema	Indica si los sistemas de control están listos	Binario
04	Carga del motor calculada	Esfuerzo del motor respecto a su capacidad total	%
0A	Presión del combustible	Mide la presión en el riel del combustible	kPa

Con el conocimiento sobre los PIDs, el proyecto se centra en uno, el 03 este es de gran valor, ya que en base a él se obtendrán los códigos de diagnóstico (DTC). Para entenderlo, cada uno de estos tiene diferente significado y se le atribuye a una posible causa. A continuación se presentan los DTC más representativos:

Código DTC	Descripción de la falla	Sistema afectado	Posible Causa
P0700	Falla en el sistema de transmisión	Transmisión	Solenoides o ECU de transmisión defectuosa
P0420	Eficiencia del catalizador por debajo del umbral	Emisiones	Catalizador dañado o sensor O2
P0172	Mezcla demasiado rica	Combustible / Aire	Inyector con exceso de combustible o sensor O2
P0110	Circuito del sensor de temperatura del aire de admisión IAT	Admisión	Mal estado en Sensor IAT o cableado
U0100	Pérdida de comunicación con la ECU principal	Red de control	Fallo en bus CAN o ECU
U0100	Pérdida de comunicación con la ECU principal	Red de control	Fallo en bus CAN o ECU
P0500	Sensor de velocidad del vehículo	Transmisión	Sensor VSS dañado

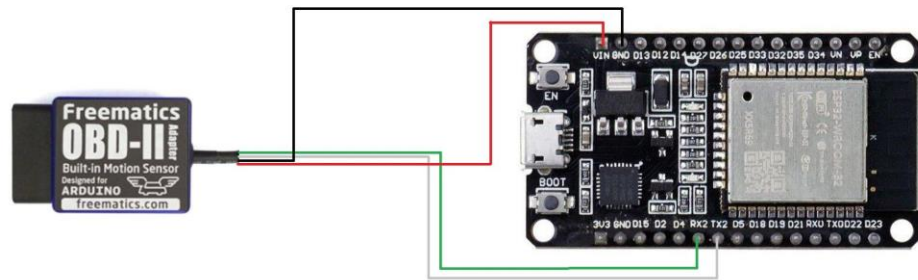
Con el dominio de los conceptos automotrices, se pasó a la implementación del sistema encargado de la recolección de los códigos DTC de los vehículos. Para ello se trabajó con el microcontrolador ESP32, herramienta encargada de procesar los datos provenientes del vehículo. Se adjunta a continuación el datasheet del microcontrolador ESP32.



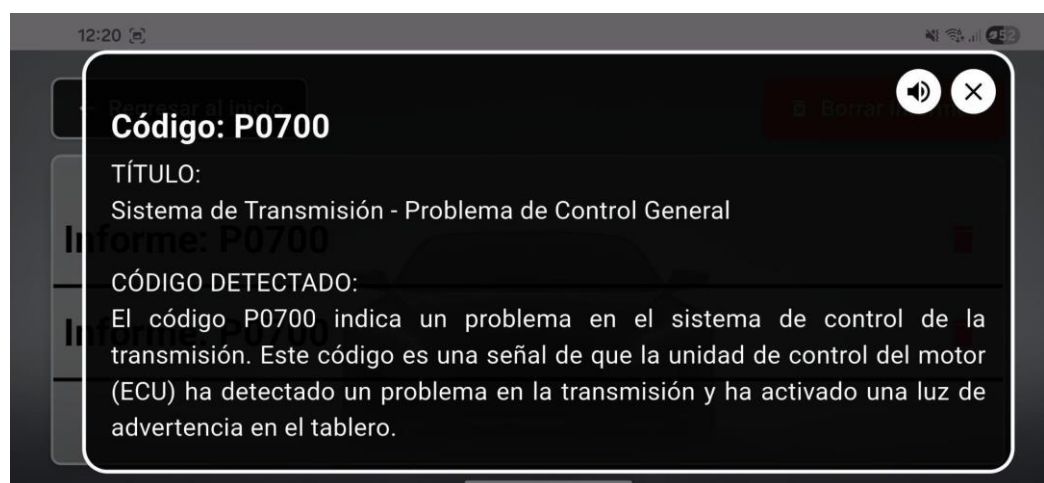
Con el microcontrolador encargado de procesar los datos del vehículo, se requirió de equipo que nos permita comunicarnos con la ECU del auto, para ello, el proyecto trabajó con el adaptador Freematics OBD-II UART Adapter. Este nos permitió trabajar en conjunto con el ESP32 por medio de las librerías proporcionadas por el fabricante del adaptador OBD-II. Se presenta el adaptador con el que se trabajó el proyecto:



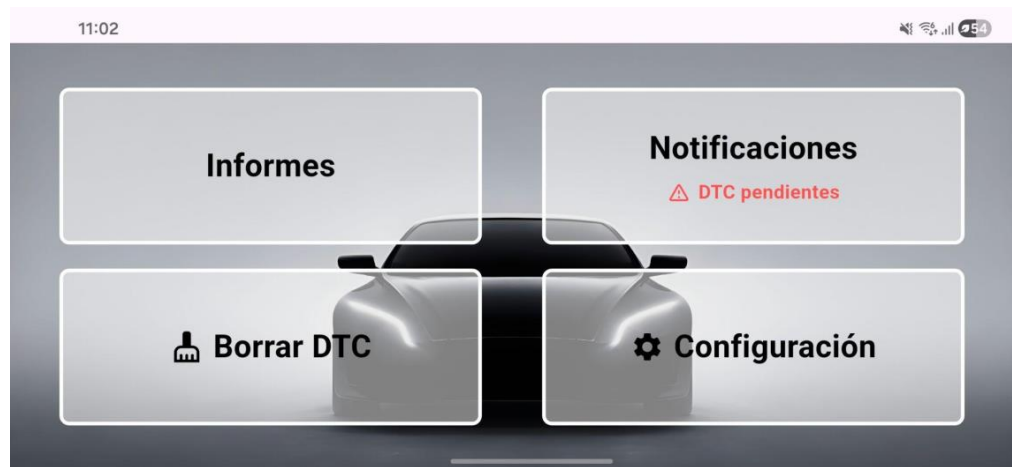
Para la implementación, se realizó la conexión del puerto RX del adaptador al puerto TX del ESP32, de igual manera, se conectó el puerto TX del adaptador al RX del ESP32. Además, el adaptador Freematics y el ESP32 compartirán GND y el adaptador alimentará el ESP32. A continuación se presenta el esquema de conexión:



Para culminar la fase de recolección de datos, se diseñó un sketch que se conecta al backend de Flask para subir los datos a Firebase, así como para recibir solicitudes de limpieza o borrado de códigos DTC. Para el aplicativo móvil se requirió de un backend que permita la integración de los datos del vehículo, como la aplicación de una API de inteligencia artificial. Es así que se completó el diseño de servicios que comuniquen tanto el ESP32 con la aplicación móvil, la base de datos de Firebase con la aplicación y la integración de la API de inteligencia artificial para el análisis y recomendaciones ante códigos DTC detectados. Para una respuesta eficaz por parte de la API de inteligencia artificial, se desarrolló un prompt que se ajusta a cada parámetro del vehículo, presentando así una respuesta más precisa y entendible para el usuario. El resultado del prompt se muestra de la siguiente manera:



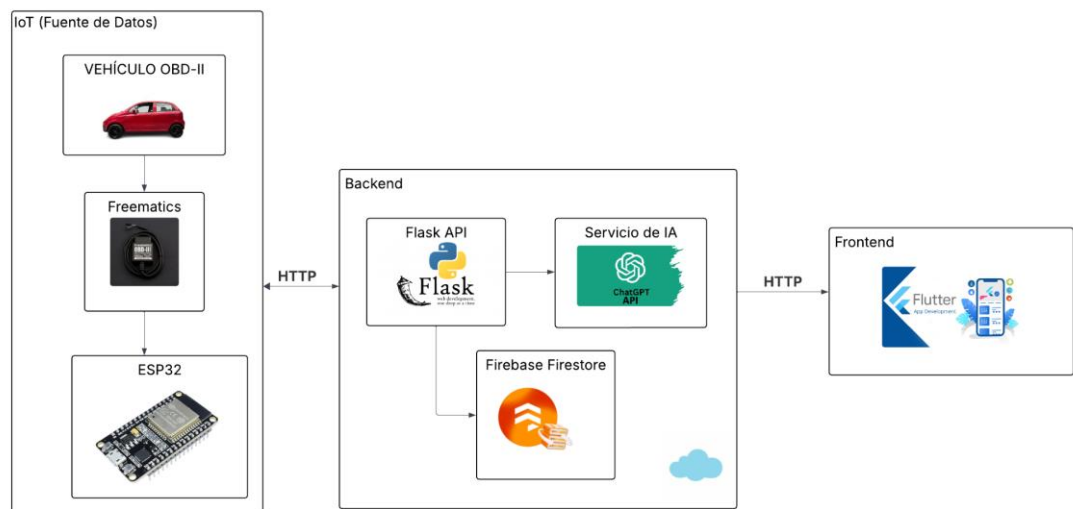
Para el aplicativo móvil se usó el framework Flutter, con el que se integró el backend y se proporcionó una interfaz intuitiva al usuario. A continuación se presenta la interfaz de la aplicación móvil:



5.2 Despliegue

Las pruebas iniciales del proyecto se realizaron con el backend desplegado de forma local, sin embargo, para su presentación, se realizó el despliegue en Render, una plataforma que permite el despliegue de servicios como lo es el backend de Flask que se desarrolló. De igual forma, en sus etapas iniciales, el aplicativo móvil se probó y trabajó con emuladores, pero para la etapa final se generó el APK para ser instalado en cualquier dispositivo Android.

A continuación se presenta la arquitectura planteada para el aplicativo de diagnóstico vehicular:



Capa de Frontend

Se implementó una interfaz gráfica en Flutter adaptable para dispositivos móviles, así como para los equipos inteligentes presentes en los vehículos. El diseño se centró en que los usuarios tengan acceso inmediato a los códigos DTC detectados, así como a las recomendaciones presentadas por la inteligencia artificial.

Capa de Backed

- Servicio Flask
 - Se expuso un conjunto de endpoints con los cuales se recibieron los datos del vehículo. Estos se almacenaron en la base de datos de Firebase Firestore y a partir de ellos se trabajó en su procesamiento para efectuar el diagnóstico vehicular.
 - Adicionalmente, a los usuarios del aplicativo se les dio la libertad de gestionar parámetros que ajusten el prompt para mejorar los resultados del diagnóstico. Los parámetros configurables son la información del vehículo, como el número de chasis, marca, modelo y año de fabricación.
 - El aplicativo le permite al usuario almacenar los resultados que se generan de los diagnósticos vehiculares. Así se garantiza un historial de diagnósticos con el cual el propietario del vehículo estará al tanto de los análisis realizados con anterioridad.
 - El servicio se encarga de gestionar la eliminación de códigos DTC en la ECU del vehículo. Se reciben solicitudes desde la aplicación móvil, estas se registran y almacenan en Firebase para que el microcontrolador ESP32 las consulte y valide y proceda con la eliminación de los DTC en la ECU del vehículo.
- Despliegue en la nube
 - El backend del proyecto se desplegó en la nube, con la mentalidad de no depender de un equipo externo y garantizar que esté disponible en todo momento de manera remota. Con el despliegue se logró integrar de forma directa el microcontrolador ESP32 con el servicio Flask, esto facilitando la comunicación entre el vehículo y el backend de la aplicación.
- Capa de IoT
 - Capa encargada de la conexión entre el vehículo y el microcontrolador ESP32 mediante el adaptador OBD-II de Freematics. Para este proceso se implementó la librería propia del fabricante, lo que facilitó la captura de la

información clave para el proyecto, en este caso, los códigos DTC. Además, los datos que capturan se transmiten al backend para ser almacenados en Firebase y ser puestos en servicio para la aplicación móvil.

5.3. Testing

Durante el desarrollo de la aplicación se realizaron pruebas para validar que cada sección del proyecto funcione correctamente. Durante el desarrollo se desarrollaron las siguientes pruebas:

- Se validó la conexión entre el adaptador de Frematics y el ESP32 con pruebas iniciales en las que se capturaron valores como RPM del motor, velocidad del vehículo y posición del acelerador.
- Se validó la captura y correcto procesamiento de códigos DTC mediante la desconexión de sensores del vehículo.
- Se validó la conexión del microcontrolador ESP32 a la red WIFI para el consumo de los endpoints del backend.
- Se validó que el microcontrolador esté cargando cada DTC bien procesado encontrado en la ECU del vehículo.
- Se verificó de forma local la correcta integración entre los endpoints y el microcontrolador previo al despliegue.
- Se verificó con la herramienta Postman que cada uno de los endpoints implementados nos dé las respuestas esperadas.
- Se revisó que la interfaz de usuario tenga una estructura sencilla y comprensible para el usuario final.
- Se revisó que la API de ChatGPT y el prompt den los resultados esperados previos a la integración con la aplicación móvil.
- Se validó que la aplicación tenga comunicación Firebase para el envío de solicitudes a la ECU para el borrado de códigos DTC.
- Se validó el despliegue del backend en Render y cómo este interactúa con el ESP32 para el envío de códigos DTC y la recepción de solicitudes de borrado de códigos DTC.
- Se verificó que los endpoints del backend sean consumidos exitosamente por la aplicación móvil, garantizando que los datos recibidos y enviados sean válidos, así como las respuestas generadas por la API de ChatGPT.

- Se verificó que el APK generado de la aplicación móvil sea compatible con los dispositivos Android, así como los sistemas inteligentes presentes en los vehículos.
- Se validó el funcionamiento de la aplicación con simulaciones de códigos DTC con ayuda de Postman para garantizar que las respuestas generadas por la API de ChatGPT sean correctas.
- Se validó el funcionamiento de la aplicación con pruebas en vehículos reales, capturando los DTC, generando el análisis y recomendaciones de los códigos DTC y la limpieza de los mismos de la ECU del vehículo.

5.4. Desarrollo con metodología en espiral

La recolección de códigos DTC mediante el adaptador Freematics OBD-II, el procesamiento de los datos a través del módulo ESP32, el almacenamiento en la nube mediante Firebase, el desarrollo de la aplicación móvil en Flutter y el uso de la API de inteligencia artificial para la generación de respuestas fueron optimizados e integrados dentro de la aplicación móvil. Esta aplicación centraliza toda la información y la presenta de manera comprensible al usuario. Bajo el enfoque de la metodología en espiral, se realizaron análisis y mejoras continuas en la arquitectura del sistema, abarcando los mecanismos de comunicación, la gestión de los datos y la interacción del usuario con el asistente de inteligencia artificial, lo que garantiza un diagnóstico vehicular eficiente y comprensible. A continuación, se detallan las mejoras realizadas para cada una de las espirales:

Primera Espiral:

- **Planificación:** Definimos como meta lograr que el dispositivo conectado al vehículo, compuesto por el ESP32 y el adaptador OBD, leyera los códigos de diagnóstico DTC y los enviara al servidor de forma segura y eficiente. Este proceso incluyó la verificación de qué códigos serían recibidos, la frecuencia de transmisión, la estabilidad de la conexión y el correcto funcionamiento del sistema en condiciones reales.
- **Análisis de Riesgos:** Los posibles problemas que surgieron fueron los siguientes: pérdida de señal Wi-Fi, errores en la comunicación entre el OBD, el ESP32 y el vehículo, o daños ocasionados por conexiones incorrectas. Para mitigar estos inconvenientes, se implementaron como soluciones la reconexión adecuada de cada

componente, la verificación de los cables y de los niveles de voltaje, asegurando que no presentaran errores; en caso de detectarlos, estos fueron registrados para su posterior revisión.

- **Desarrollo:** Para el desarrollo se realizó un primer programa en el cual el ESP32 leyó los datos y los envió al servidor. Asimismo, se llevaron a cabo pruebas básicas con datos simulados y, posteriormente, se utilizó el conector OBD conectado al vehículo en condiciones controladas, específicamente en un estacionamiento. Finalmente, se registraron y corrigieron los errores que fueron detectados.
- **Evaluación:** Se verificó que la planificación se haya cumplido, que los datos fueran recibidos de forma segura y sin errores que representaran problemas graves. Una vez confirmados estos aspectos, se documentó lo logrado y se planificó la siguiente espiral.

Las actividades realizadas en esta espiral fueron:

El adaptador Freematics OBD-II permitió extraer los códigos DTC del vehículo, facilitando el acceso a dicha información clave para el desarrollo del sistema. Para ello se utilizó el entorno de Arduino IDE, donde se implementó un sistema integrado basado en el ESP32 que conecta la ECU mediante comunicación OBD-II por UART. El ESP32 establece una conexión WiFi y envía periódicamente los códigos de error en formato JSON al servidor de Flask, encargado de almacenarlos en la base de datos.

Durante esta etapa se identificaron consideraciones técnicas relacionadas con la compatibilidad entre el ESP32 y el adaptador OBD-II, las cuales fueron resueltas mediante el uso de librerías específicas en el Arduino IDE. Además, se implementó una función para el borrado de códigos DTC bajo solicitud remota. Por lo tanto, se concluyó satisfactoriamente la primera espiral y permitió sentar las bases para las siguientes etapas del proyecto.

Segunda Espiral:

- **Planificación:** La meta alcanzada fue contar con un servidor, como Flask, que recibió los datos provenientes del vehículo y los almacenó en la nube de Firebase. Además, se preparó la integración con una capa de inteligencia artificial, como ChatGPT, la cual fue capaz de interpretar los códigos del vehículo y generar

explicaciones comprensibles. En esta etapa también se definió el despliegue en la nube, el cual permitió que los servicios y aplicaciones del sistema operaran directamente sobre plataformas de computación en la nube.

- **Análisis de Riesgos:** Se estudiaron riesgos como los costos asociados al uso de la nube, el consumo de la API de inteligencia artificial, la utilización de un prompt mal construido, la lentitud en las respuestas y la pérdida de la señal Wi-Fi. Para mitigar estos riesgos, se implementaron estados que permitieron identificar situaciones como “cargando” o “sin conexión”, con el fin de limitar actualizaciones o consultas innecesarias y reducir el uso de la nube o de la inteligencia artificial. Además, se evaluó la correcta configuración del entorno de despliegue en la nube, evitando vulnerabilidades o interrupciones en la disponibilidad del servicio.
- **Desarrollo:** Se crearon y configuraron los endpoints necesarios en el servidor para recibir y consultar datos, junto con la conexión a la base de datos alojada en la nube. Asimismo, se realizaron pruebas con datos simulados para verificar que el envío y el almacenamiento de la información se efectuaran correctamente. Como parte del despliegue en la nube, los servicios del sistema, incluyendo las APIs encargadas de recibir los datos del ESP32, los procesos de almacenamiento en la base de datos y las llamadas a la inteligencia artificial fueron alojados en plataformas escalables y accesibles desde cualquier lugar. Finalmente, se añadió la primera versión de la función de consulta de la inteligencia artificial para la interpretación de códigos.
- **Evaluación:** Se comprobó que tanto el servidor como la base de datos de Firebase almacenaron los datos correctamente y sin fallos, y que la integración con la inteligencia artificial, así como sus respuestas, ofrecieron explicaciones razonables y acordes al tema tratado. Asimismo, se verificó que el despliegue en la nube garantizó la disponibilidad, la seguridad y el correcto funcionamiento de los servicios. De igual manera, se documentaron los fallos, se corrigieron y se planificaron mejoras para la siguiente iteración.

Las actividades realizadas en esta espiral fueron:

En el transcurso de la segunda espiral, se desarrollaron varios endpoints en el servidor backend, que permitieron la recepción de los datos desde el vehículo y que se utilizaron en la aplicación móvil desarrollada en Flutter. Además, se implementó el envío y almacenamiento de la información en la nube usando Firebase. En el

marco de esta etapa, se integró una API de inteligencia artificial (ChatGPT), para lo cual se diseñó y se ajustó un prompt que permita interpretar los códigos DTC y desarrollar explicaciones comprensibles para el usuario.

El backend completo fue desplegado en la nube Render, utilizando su plan gratuito. Durante este proceso se identificaron consideraciones técnicas relacionadas con el uso de la API de inteligencia artificial, como las limitaciones en la frecuencia de consultas debido a costos asociados, lo que motivó la optimización del uso de la IA y el ajuste del prompt para garantizar respuestas correctas y recomendaciones generales adecuadas. Asimismo, se evidenciaron limitaciones propias del plan gratuito del servidor de Render, tales como una mayor latencia en la primera solicitud y diferencias en la zona horaria del servidor, ubicado en Norteamérica, lo que afecta el registro de fecha y hora de los datos.

Las limitaciones mencionadas no corresponden a fallos del sistema, sino a restricciones externas relacionadas con herramientas como Render y la API de ChatGPT. Estos inconvenientes pueden resolverse mediante la asignación de recursos económicos, ya sea para aumentar el número de consultas o acceder a un plan superior. Por ello, se considera que los objetivos de la segunda espiral fueron alcanzados y permitieron avanzar hacia la etapa final del desarrollo del sistema.

Tercera Espiral:

- **Planificación:** El objetivo fue crear una aplicación móvil en Flutter que mostró al usuario los datos del vehículo en tiempo real. Asimismo, permitió consultar al asistente de inteligencia artificial para obtener explicaciones y recomendaciones sobre dichos datos. En la aplicación se definió la estructura y los componentes, tales como el tablero de reportes de códigos, las notificaciones y el uso de la inteligencia artificial.
- **Análisis de riesgos:** Se identificaron problemas como una interfaz confusa para el usuario, poco intuitiva o con componentes mal posicionados, así como el uso excesivo de datos por parte de la aplicación y el retraso en la información. Por ello, la solución implementada fue diseñar pantallas claras y fáciles de entender, mostrar estados que permitieron al usuario conocer lo que ocurría dentro de la aplicación y establecer límites en las actualizaciones innecesarias para optimizar el uso de lecturas en la nube.

- **Desarrollo:** Se desarrolló una versión mínima de la aplicación en la cual se mostraron los datos y se permitió la consulta a la inteligencia artificial. Asimismo, se realizaron pruebas en teléfonos reales para verificar que la aplicación respondiera correctamente, que la información se mantuviera actualizada y que la experiencia de usuario fuera satisfactoria incluso para personas sin conocimientos técnicos.
- **Evaluación:** Se recopilaron pruebas de uso y se evaluó la facilidad de comprensión de la aplicación. Posteriormente, se determinó si la aplicación estaba lista para su uso o si fue necesario realizar ajustes en el rendimiento y la usabilidad, lo que permitió identificar que la aplicación cumplió con todos los objetivos planteados y ofreció una experiencia adecuada para todos los usuarios.

Las actividades realizadas en esta espiral fueron:

Durante la tercera espiral se desarrolló la aplicación móvil en Flutter, con una interfaz simple y clara para todos los usuarios. Fue diseñada con elementos grandes y una orientación horizontal obligatoria, de modo que ocupen el mayor espacio en la pantalla y que la información sea visible tanto en dispositivos pequeños como en grandes.

La aplicación integra cuatro funcionalidades principales: el panel de reportes, donde se pueden visualizar todas las consultas generadas por el usuario mediante el uso de la inteligencia artificial; el panel de notificaciones, en el que los códigos registrados en la base de datos aparecen automáticamente con su fecha y hora y al hacer clic en una notificación se abre el asistente de inteligencia artificial que ofrece explicaciones detalladas sobre el código detectado, además se agrega la función de lectura en voz alta, que permite a los usuarios escuchar la información proporcionada por la IA mientras realizan otras tareas.

La tercera funcionalidad permite borrar códigos DTC mediante el envío de instrucciones a la ECU del vehículo, con el objetivo de eliminar aquellos códigos asociados a fallos menores o generales. Finalmente, la cuarta funcionalidad corresponde al panel de configuración, que permite al usuario registrar y editar parámetros del vehículo como marca, modelo, año y número de chasis (VIN). En el primer uso de la aplicación, esta información se solicita de manera obligatoria.

Asimismo, cuando la aplicación se encuentra en segundo plano o no está abierta, el sistema alerta al usuario sobre los nuevos códigos DTC registrados mediante notificaciones emergentes. Para ello, se implementó Firebase Cloud Messaging (FCM), encargado de enviar notificaciones push a los dispositivos móviles.

Durante el desarrollo se presentaron desafíos técnicos, como el aprendizaje inicial del framework Flutter, la adaptación de la interfaz a pantallas de gran tamaño y la configuración de permisos necesarios para el correcto funcionamiento de las notificaciones. Estos aspectos fueron superados mediante el estudio de la tecnología, la reorganización del diseño visual y la adecuada configuración de los servicios de Firebase. En consecuencia, los objetivos de la tercera y última espiral se cumplieron satisfactoriamente, concluyendo el desarrollo del sistema propuesto.

6. Resultados

6.1 Resultados del sistema en un Chevrolet Spark Life

El sistema que se ha realizado se evaluó en un Chevrolet Spark Life STD 1.0, 5 puertas, 4x2, correspondiente al modelo 2019.

Para esta demostración fue necesario provocar un código de error (DTC), ya que el vehículo utilizado no presentaba fallas que pudieran generar estos códigos. Con este fin, se generó un DTC desconectando el sensor MAP. Este sensor fue seleccionado debido a que resulta sencillo de retirar y reinstalar. Además, su desconexión activa la luz de check engine en el tablero, lo cual es útil para verificar el funcionamiento del sistema. Una vez obtenido este código, se procedió a realizar las pruebas correspondientes.



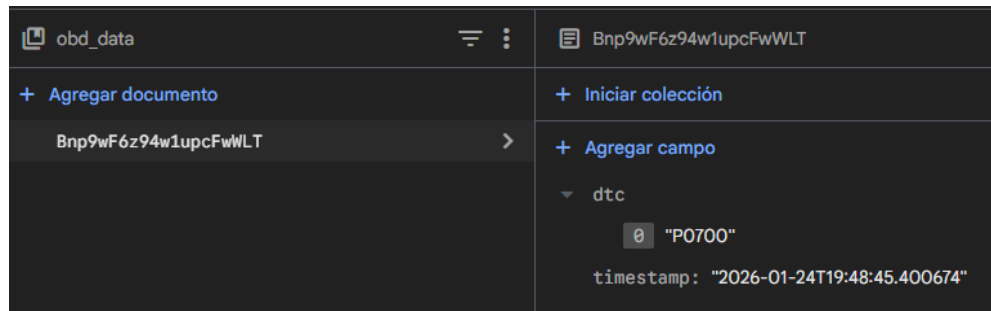
Con el fin de obtener los códigos de la ECU del vehículo, primero se debe poner el vehículo en modo ON, ya que en este estado la computadora tiene energía y habilita la comunicación entre los sensores y el adaptador. A continuación se debe localizar el puerto OBD-II, que generalmente se encuentra debajo del tablero, cerca de la columna de dirección, por lo que una vez identificado se procede a conectar el adaptador Freematics OBD-II. Al momento que se realiza la conexión el adaptador enciende una luz azul que nos indica que el dispositivo ha recibido energía y está listo para establecer la comunicación con la ECU, Con esta confirmación, se puede continuar con la lectura de los códigos.



Gracias al microcontrolador ESP32 es posible verificar los códigos que se reciben a través del adaptador Freematics OBD-II. Estos códigos se visualizan en la consola del Arduino IDE y, de manera similar, se pueden observar distintos estados del ESP32, como su conexión a la red Wi-Fi, la correcta comunicación con el OBD-II y el envío de los códigos mediante solicitudes HTTP hacia la base de datos de Firebase.

Como podemos observar, el código que el adaptador Freematics OBD-II envió es el código DTC: P0700. Este código fue el resultado de desconectar el sensor MAP del vehículo. De la misma manera, podemos observar que el microcontrolador ESP32 envió correctamente el código a la base de datos de Firebase.

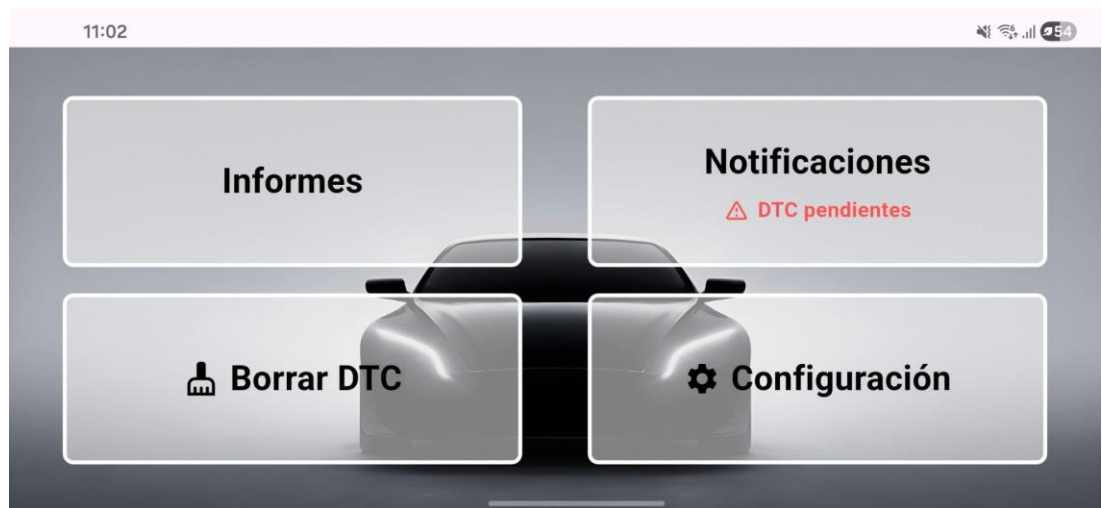
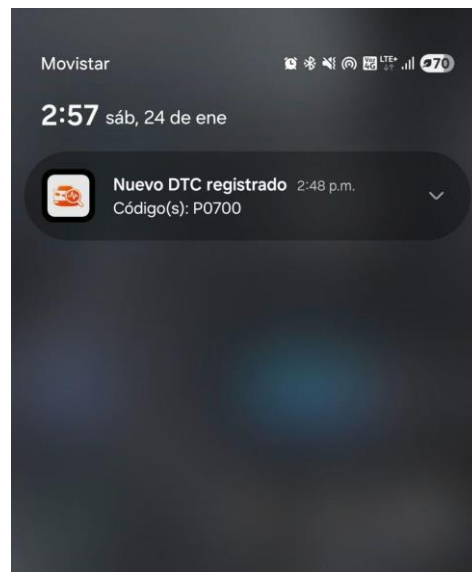
```
Conectando WiFi...
WiFi conectado
10.95.199.244
OBD listo
HTTP GET /status -> 200
DTC #1 | Raw: 0x700 -> DTC: P0700 | Válido: SI
HTTP POST /obd -> 200
```



Si es la primera vez que se instala la aplicación, esta solicita ingresar los datos del vehículo, como la marca, el modelo, el año y el número de chasis o VIN. Estos datos son fundamentales, ya que permiten alimentar las respuestas generadas por la inteligencia artificial.

A screenshot of a mobile application form. The form has four rows, each with a label on the left and a text input field on the right. The labels are 'Marca:', 'Modelo:', 'Año:', and 'Número VIN:'. The input fields contain the following text: 'SPARK LIFE STD 1.0 5P 4X2 TM', 'Chevrolet', '2019', and '9GAMM610XKB003013'. The form is displayed on a mobile device screen, with a status bar at the top showing the time '2:49' and various icons.

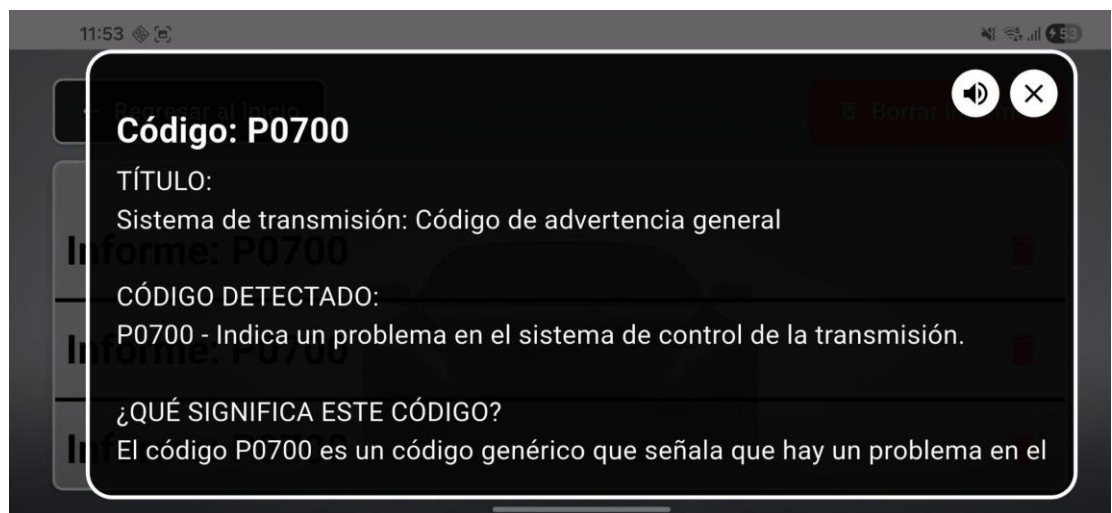
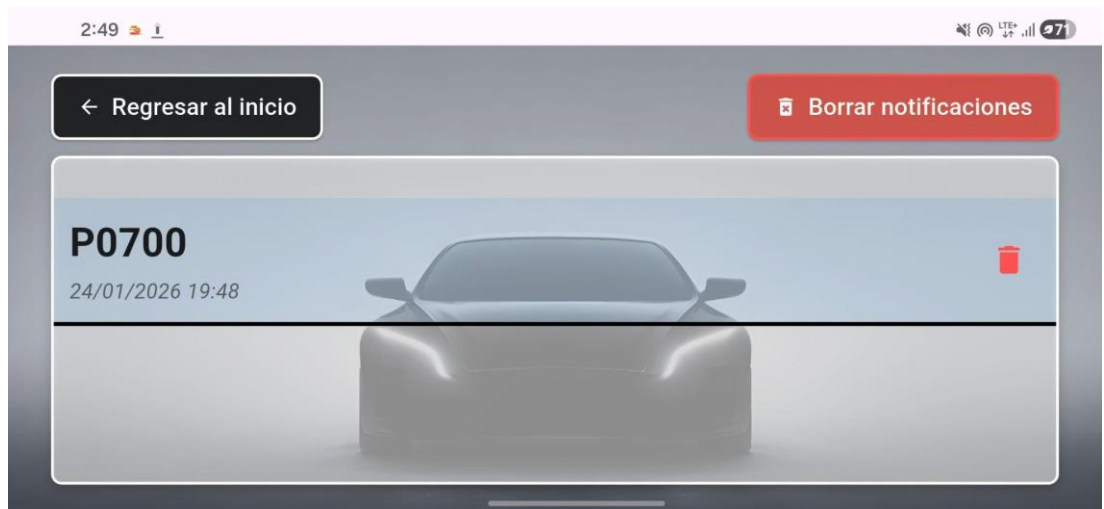
Dentro de la aplicación móvil se puede visualizar en tiempo real el código registrado en la base de datos, lo que permite verificar que la comunicación entre el adaptador OBD-II, el microcontrolador ESP32 y el servidor en Firebase se ha realizado de manera correcta. De la misma manera, si el usuario no tiene abierta la aplicación, el sistema envía un mensaje pop-up que indica que se ha registrado un nuevo código, lo que garantiza que el usuario reciba la notificación inmediata de la falla detectada en el vehículo.



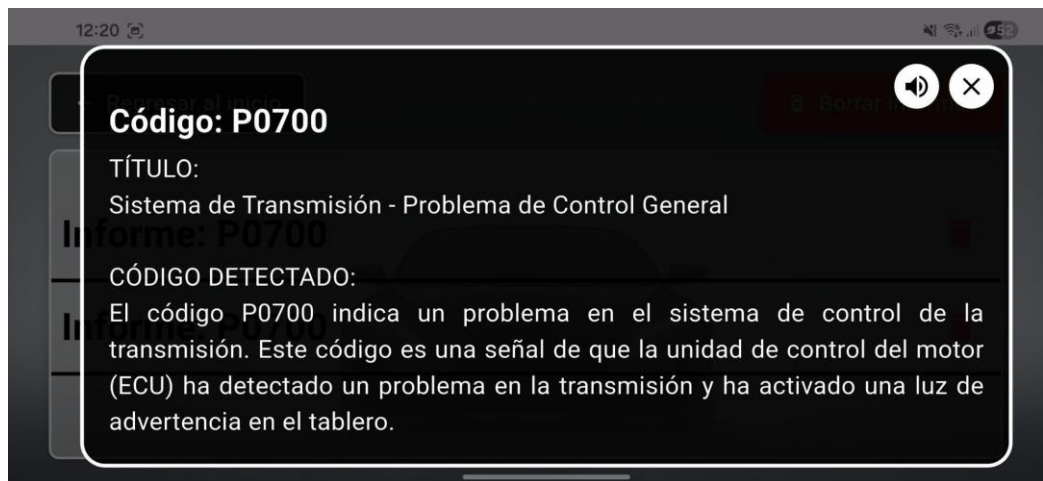
En el panel de notificaciones podemos observar el código registrado junto con su fecha y hora. De esta manera, el usuario podrá identificar con precisión cuándo ocurrió la falla y mantener un historial ordenado de los eventos. Al hacer clic en la notificación se desplegará la consulta a la IA, la cual proporcionará toda la información necesaria acerca del código, como explicar qué significa, si representa una falla grave o leve, y ofrecer enlaces relacionados con la pieza del vehículo dañada, además de posibles soluciones o recomendaciones de mantenimiento.

El usuario podrá realizar consultas tantas veces como lo desee del mismo código, generando nuevas respuestas en caso de que la primera no sea satisfactoria. Además, contará con una función para escuchar la consulta de la IA en voz alta. Esta función resulta útil cuando el conductor está ocupado, desea evitar distracciones visuales o prefiere recibir la información de manera auditiva mientras realiza otras tareas. Por último, el panel de

notificaciones ofrece la opción de eliminar cada notificación de forma individual o, si lo desea, borrar todas las notificaciones al mismo tiempo, lo que permite mantener el sistema organizado y libre de registros innecesarios.



Por otro lado, en el panel de informes se pueden ver los registros generados a partir de las consultas realizadas en la sección de notificaciones. Al hacer clic en cada informe, se despliega la consulta correspondiente, con la opción de lectura en voz alta. Además, el panel permite eliminar informes de manera individual o borrar todos al mismo tiempo.

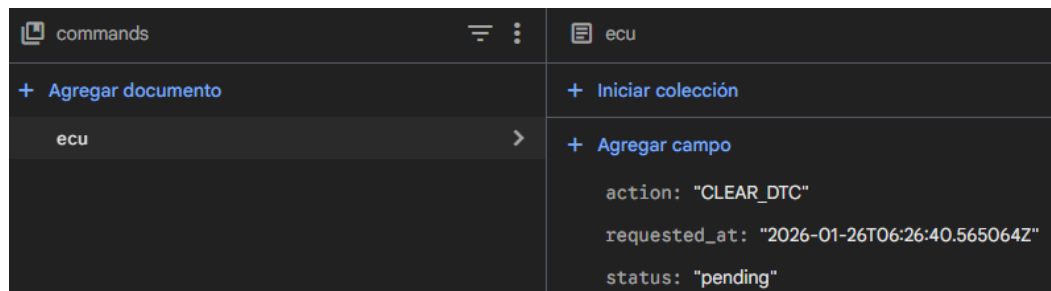
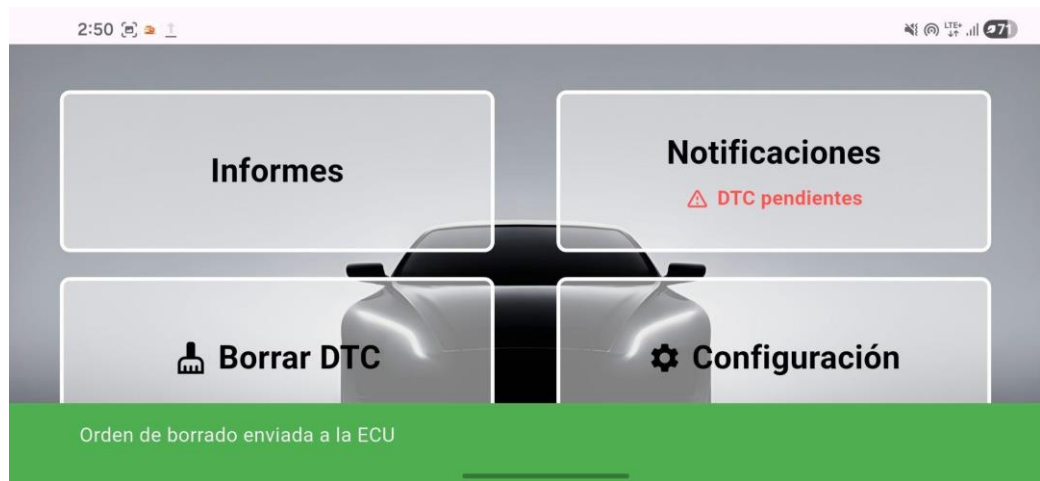


El panel de Configuración permite al usuario cambiar o actualizar los datos de su vehículo, por ejemplo, reemplazarlos con la información de otro vehículo.



En el panel principal se dispone de la función Borrar DTC, la cual, al ser activada por el usuario, muestra un mensaje de confirmación y registra una solicitud en la base de datos Firebase. El ESP32 consulta periódicamente dicha base y, al detectar una acción pendiente

de tipo CLEAR_DTC, envía el comando correspondiente a través del adaptador Freematics OBD-II a la ECU del vehículo para eliminar los códigos de falla almacenados. De esta manera, se pueden eliminar códigos asociados a fallas menores o generales, lo que puede provocar el apagado de la luz check engine en el tablero del vehículo. Así, se concluye que el sistema cumple su función de mostrar los códigos DTC, informar a los usuarios sobre ellos mediante el uso de inteligencia artificial y enviar comandos para su eliminación.

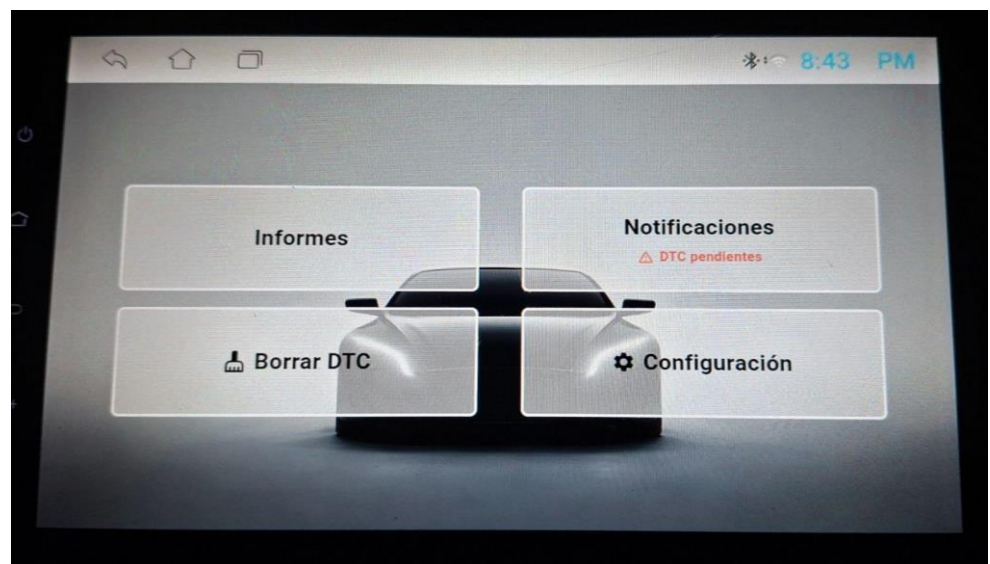


```
HTTP GET /status -> 200
Ejecutando MODE 04 (Clear DTC)
HTTP POST /confirm -> 200
```

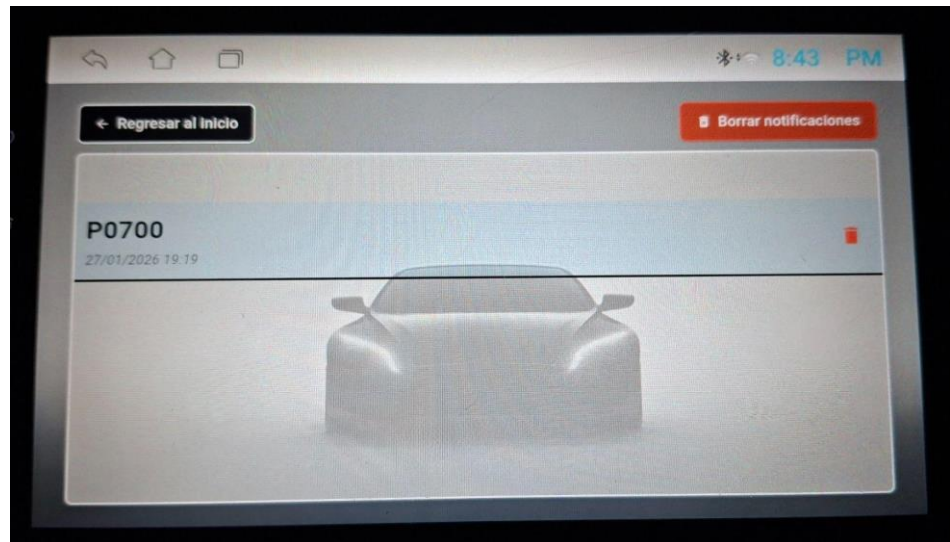



Por último, este sistema fue probado en radios Android con el objetivo de que los usuarios puedan interactuar directamente desde la interfaz del vehículo, sin necesidad de dispositivos externos, ya que se adapta a cualquier tipo de pantalla que el usuario desee utilizar. A continuación se muestra el resultado:

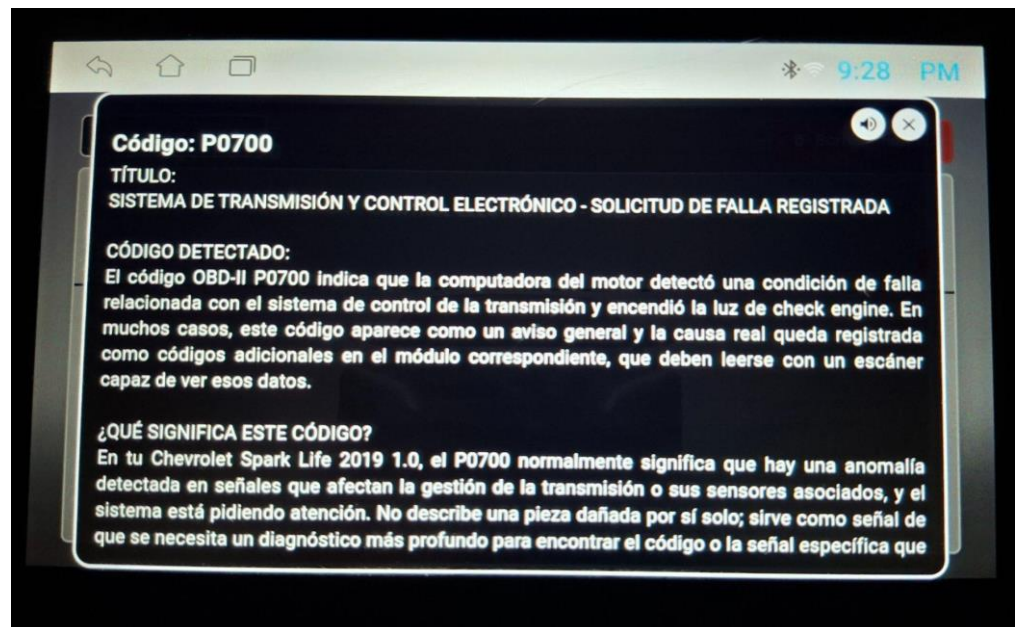
- Panel principal con las distintas funciones.



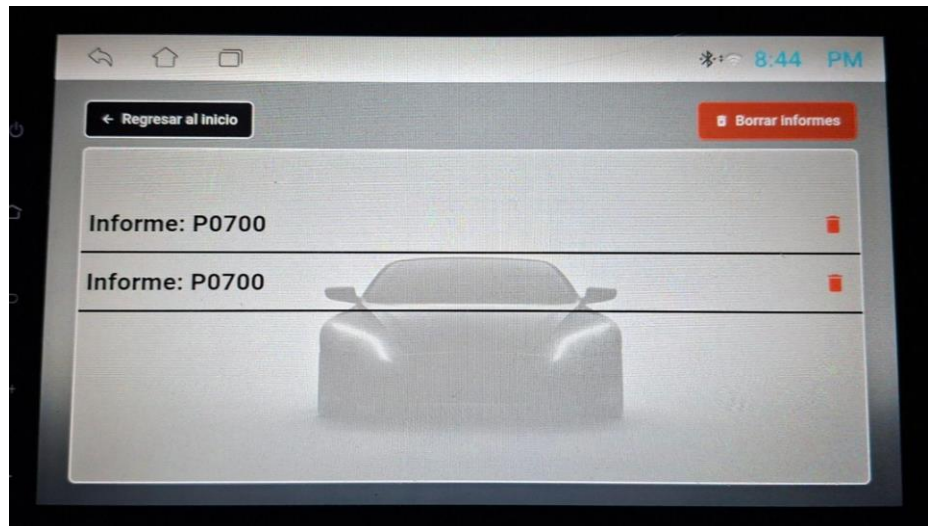
- Panel de notificaciones.



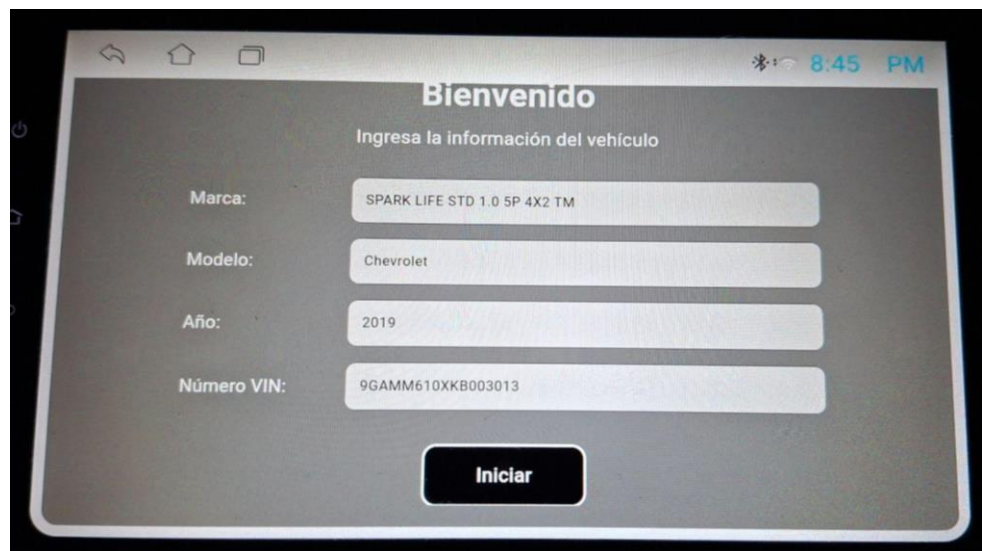
- Consulta a la IA con función de lectura en voz alta.



- Panel de informes.



- Configuración del vehículo.



6.2 Resultados del sistema en una Great Wall Wingle

Se realizó la prueba en otro vehículo, en concreto en una camioneta Great Wall Wingle año 2013, y los resultados fueron los siguientes: para comprobar el correcto funcionamiento del sistema, se utilizó un vehículo que presentaba códigos de error por fallas naturales y no provocadas. En este escenario, la camioneta mostraba varias fallas mecánicas que pudieron ser detectadas mediante el adaptador y la aplicación móvil.

Los pasos para el funcionamiento del sistema fueron los mismos que se utilizaron en el Chevrolet Spark, por lo que primero fue necesario ingresar la información del vehículo correspondiente.

11:53

Marca: Great Wall

Modelo: Wingle

Año: 2013

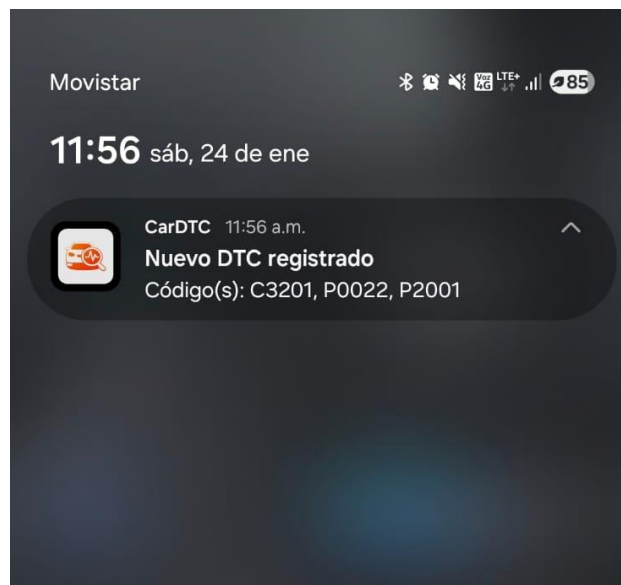
Número VIN: 8L4CA2175DC000055

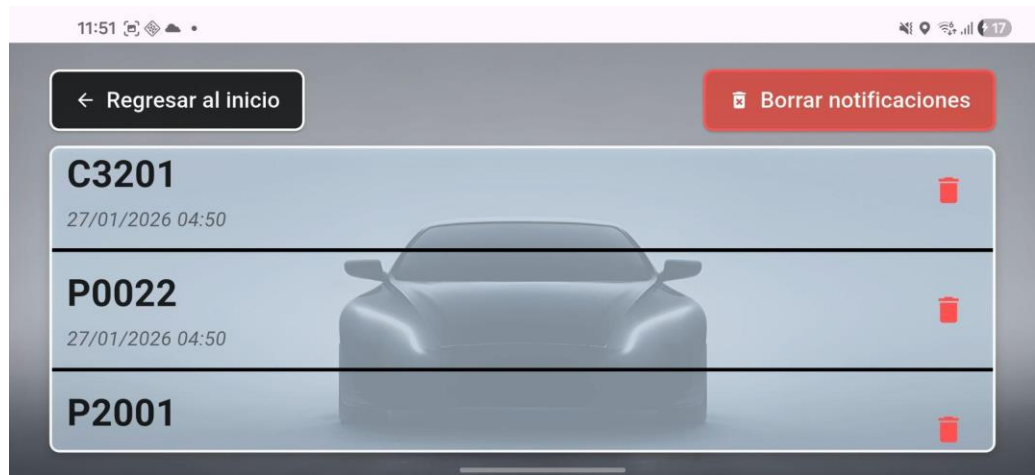
Una vez ingresados los datos, se procede a conectar el adaptador Freematics OBD-II y el ESP32 al vehículo.



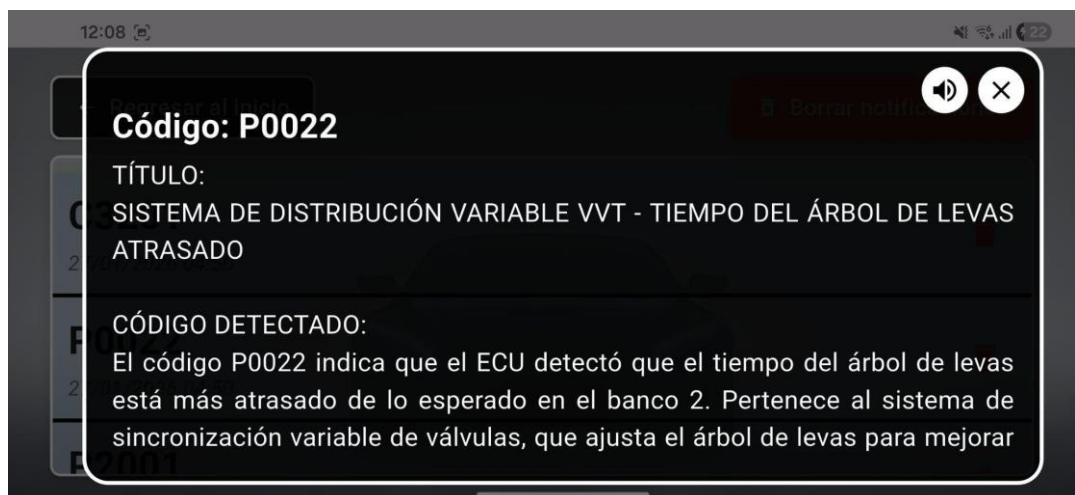


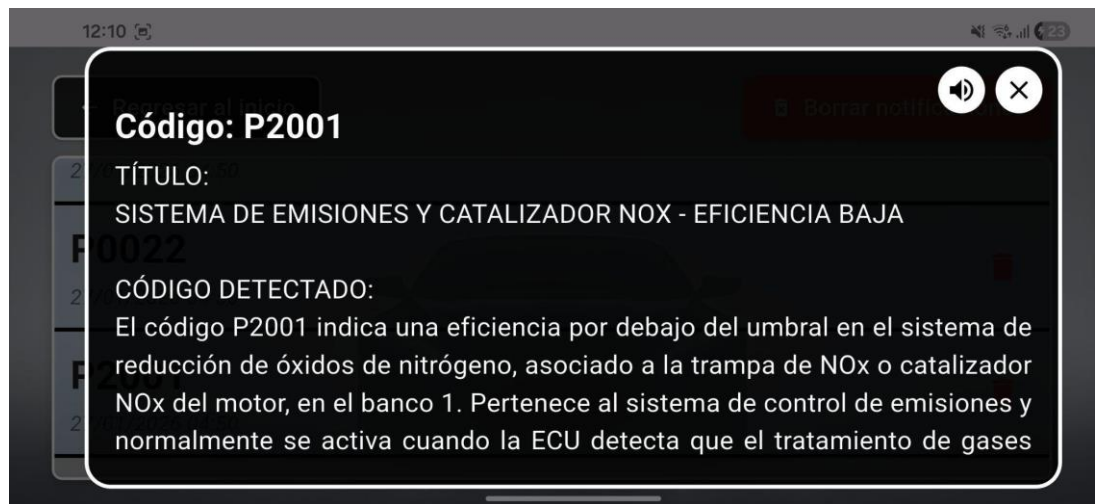
Después de conectar, la aplicación mostró tres códigos que presentaba el vehículo. Estos fueron notificados mediante mensajes pop-up y registrados en el panel de notificaciones.





De la misma manera, realizamos consultas a la IA de los tres códigos obtenidos y probamos la función de lectura en voz alta en cada una de ellas.





En el panel de informes podemos observar las tres consultas realizadas a la IA desde el panel de notificaciones.



Con esto podemos concluir que el sistema funciona de manera correcta en automóviles que cumplen con el estándar OBD-II y utilizan el protocolo ISO 15765-4 sobre la red CAN, lo que garantiza una amplia compatibilidad entre marcas y modelos. Para esta prueba se omitió la función de Borrar DTC debido a que no se contó con la autorización necesaria, ya que el vehículo utilizado pertenecía a un tercero; aun así, el sistema demostró su capacidad para detectar, registrar y notificar códigos de falla, ofreciendo a los usuarios una herramienta confiable para el diagnóstico mecánico.

7. Cronograma

OE1: Analizar y sistematizar la información técnica relacionada con los protocolos de comunicación OBD-II empleados por diferentes marcas y modelos de vehículos, mediante

la revisión de documentación técnica y bibliografía especializada, con el fin de establecer los requerimientos funcionales del sistema.

No.	Actividad
1	Realizar una investigación sobre los estándares OBD-II y sus variantes, recopilando información técnica para el entendimiento de sus características y diferencias.
2	Elaborar una matriz comparativa para los protocolos OBD-II utilizados por las principales marcas y modelos de automóviles, para la identificación de parámetros clave para el desarrollo de la aplicación.
3	Elaborar un informe que consolide hallazgos y establezca los requerimientos para el sistema de diagnóstico vehicular.

Objetivo	Actividad	Fecha inicio	Fecha fin	Horas por responsable	Responsables
OE1	Ac. 1	10/11/2025	12/11/2025	30	Juan Quizhpi Franklin Guapisaca
OE1	Ac. 2	13/11/2025	15/11/2025	30	Juan Quizhpi Franklin Guapisaca
OE1	Ac. 3	16/11/2025	17/11/2025	15	Juan Quizhpi Franklin Guapisaca

OE2: Implementar un sistema embebido que establezca comunicación con la unidad de control electrónico (ECU) del vehículo a través de la interfaz OBD-II, encargado de recopilar, procesar y transmitir los datos de diagnóstico hacia la aplicación móvil, garantizando la correcta interpretación de los parámetros automotrices.

No.	Actividad
4	Conexión y lectura de datos OBD-II desde el módulo ESP32
5	Procesamiento y empaquetado de los códigos DTCs del vehículo
6	Transmisión de datos hacia Firestore por medio de Flask, previo a la conexión con la aplicación Móvil.

Objetivo	Actividad	Fecha inicio	Fecha fin	Horas por responsable	Responsables
OE2	Ac. 4	18/11/2025	21/11/2025	50	Juan Quizhpi Franklin Guapisaca
OE2	Ac. 5	22/11/2025	25/11/2025	50	Juan Quizhpi Franklin Guapisaca
OE2	Ac. 6	26/11/2025	28/11/2025	40	Juan Quizhpi Franklin Guapisaca

OE3: Implementar la conexión con herramientas de inteligencia artificial para el análisis de los datos obtenidos del sistema OBD-II, con el fin de identificar patrones, predecir fallas y generar recomendaciones de mantenimiento preventivo.

No.	Actividad
7	Diseño del prompt ajustado para el diagnóstico vehicular e implementación de la API de ChatGPT.

Objetivo	Actividad	Fecha inicio	Fecha fin	Horas por responsable	Responsables
OE3	Ac. 7	29/11/2025	2/12/2025	60	Juan Quizhpi Franklin Guapisaca

OE4: Desarrollar una aplicación móvil para Android que reciba, procese y visualice en tiempo real la información obtenida del sistema embebido, mediante una interfaz gráfica intuitiva y el uso de herramientas modernas de desarrollo móvil, facilitando el monitoreo y diagnóstico de fallas vehiculares.

No.	Actividad
8	Diseñar y desarrollar la interfaz móvil en Flutter que permita la recepción, procesamiento y visualización en tiempo real de los datos provenientes del sistema embebido.

Objetivo	Actividad	Fecha inicio	Fecha fin	Horas por responsable	Responsables
OE4	Ac. 8	3/12/2025	19/12/2025	150	Juan Quizhpi Franklin Guapisaca

OE5: Validar el funcionamiento y desempeño del sistema propuesto mediante pruebas experimentales en diferentes vehículos, evaluando la precisión de los diagnósticos, la eficiencia de la comunicación y la efectividad de los servicios de inteligencia artificial utilizados.

No.	Actividad
-----	-----------

9	Realizar pruebas en el aplicativo móvil para validar cada una de las funcionalidades que ofrece.
10	Realizar pruebas en diferentes vehículos para validar el desempeño del diagnóstico vehicular, considerando la precisión de la respuesta.

Objetivo	Actividad	Fecha inicio	Fecha fin	Horas por responsable	Responsables
OE5	Ac. 9	20/12/2025	28/12/2025	30	Juan Quizhpi Franklin Guapisaca
OE5	Ac. 10	29/12/2025	5/1/2026	25	Juan Quizhpi Franklin Guapisaca

8. Presupuesto

DENOMINACIÓN	CANT.	COSTO UNITARIO	COSTO TOTAL
	Unidades	Dólares	Dólares
1. Bienes			
ESP32	1	10.00	10.00
Freematics OBD-II UART Adapter V2.1	1	70.00	70.00
Paquete Cables Jumper	1	6.25	6.25
Resistencias	7	0.15	1.05

Fuente Automotriz	1	20.00	20.00
Cable USB	1	5.00	5.00
2. Tecnológico			
Computador portátil	2	1500.00	3000.00
3. Servicios			
API de open AI	1	20.00	20.00
Datos Móviles	1	15.00	15.00
4. Personal			
Estudiante / Desarrollador	480 horas (2 estudiantes)	15 por hora	7200.00
5. Otros			
Imprevistos	1	50.00	50.00
TOTAL			10397.30

9. Conclusiones

El desarrollo del proyecto permitió implementar un sistema que sea un soporte para los propietarios de vehículos al ofrecerles un diagnóstico vehicular en un lenguaje comprensible. Para ello se integró dispositivos embebidos, servicios de backend e infraestructura en la nube. A partir del adaptador Freematics OBD-II UART y el microcontrolador ESP32, se logró la comunicación con la ECU del vehículo, siendo así capturado cada código DTC útil para el diagnóstico vehicular.

Con la construcción de la capa de backend mediante el uso de Flask, se gestionó toda la información proveniente del vehículo. Mediante los endpoints REST, nuestro proyecto recibió, procesó y almacenó los datos en Firebase Firestore, lo que aseguró la persistencia, escalabilidad y disponibilidad de los datos para el diagnóstico vehicular. Trabajando bajo este enfoque, se integró exitosamente el hardware encargado de la adquisición de los datos y la base de datos en la nube.

Desde el comienzo se consideró el despliegue del backend en la nube para eliminar la dependencia de un equipo local para desplegar nuestros endpoints REST. Siendo así, nuestro backend se aloja en Render, plataforma para despliegue de servicios web. Gracias a esto, nuestro microcontrolador EPS32 y el adaptador OBD-II interactúan de forma remota con el servidor, lo que nos garantiza que los datos del vehículo se envían continuamente para su respectivo análisis y diagnóstico. Esta integración expone el alcance de nuestro proyecto como una solución viable para entornos reales a los que se adaptaría sin problema.

La implementación de la lectura y eliminación de códigos DTC validó el funcionamiento del sistema frente a diferentes vehículos compatibles con el estándar OBD-II. Además, el aplicativo móvil presenta resultados muy relevantes en la identificación de fallas en el vehículo, así como la recomendación con información útil para el diagnóstico preventivo y correctivo.

Finalmente, la integración de todas las capas del sistema permitió el desarrollo de una solución robusta, funcional, modular y escalable. Con este trabajo se sienta una base para mejoras futuras, en donde se plantea la incorporación de un chatbot que asesore al usuario por medio del conocimiento del manual del vehículo, la incorporación de análisis más avanzados y la visualización de información de alto valor para el diagnóstico vehicular. Este proyecto contribuye al desarrollo de herramientas tecnológicas orientadas al apoyo informático para un mejor mantenimiento y diagnóstico vehicular.

10. Recomendaciones

- Se recomienda que, para futuros trabajos que se relacionen con el diagnóstico vehicular, se trabaje primero en el estudio y dominio del campo automotriz, en especial que se abarque los conceptos clave sobre protocolo OBD-II, funcionamiento de la ECU e interpretación de códigos DTC. De esta manera se facilita la integración de los conocimientos del campo de computación y automotriz
- Para mejorar la compatibilidad en los vehículos, se plantea el uso de adaptadores OBD-II que tengan soporte para la mayor cantidad de protocolos OBD-II. Es así que el rango de vehículos para realizar el diagnóstico inteligente aumenta, ya que se eliminan las limitaciones presentes por las diferencias que existen entre los fabricantes automotrices.
- Debido a la dificultad de generar códigos DTC reales durante la etapa de pruebas, se recomienda evaluar al sistema mediante simulaciones o entornos que puedan emular fallas vehiculares. De esta manera se valida que el sistema se adapta a distintos escenarios sin depender de vehículos reales.

- Para la generación de la explicación para el usuario, se recomienda mejorar el diseño y ajuste del prompt. En particular, se considera enriquecer al prompt con información más específica en base al modelo y marca del vehículo. De esta manera, la interpretación de códigos DTC genéricos y poco descriptivos serán de más utilidad para el usuario que busca una explicación de qué le sucede a su vehículo.

11. Uso de IA Generativa (GenAI)

GenAI ha facilitado la redacción y el refinamiento del lenguaje, pero el desarrollo de ideas, argumentos, resultados y conclusiones académicas/científicas se han realizado por el/los autores. Todo el contenido generado por GenAI ha sido revisado y editado minuciosamente por el/los autores, quienes asumen toda la responsabilidad por el contenido final de esta publicación.

12. Referencias bibliográficas

- Arrobo Carrión, J. C. (2021). Identificación y evaluación de riesgos mecánicos en el mantenimiento automotriz de la flota vehicular de una empresa prestadora de servicios para el sector minero, en la provincia de Zamora Chinchipe-Ecuador en el año 2021.
- Mahale, Y., Kolhar, S. & More, A. Enhancing predictive maintenance in automotive industry: addressing class imbalance using advanced machine learning techniques. *Discov Appl Sci* 7, 340 (2025). <https://doi.org/10.1007/s42452-025-06827-3>
- Michailidis, Emmanouel & Panagiotopoulou, Antigoni & Papadakis, Andreas. (2025). A Review of OBD-II-Based Machine Learning Applications for Sustainable, Efficient, Secure, and Safe Vehicle Driving. *Sensors*. 25. 4057. 10.3390/s25134057.
- Blasco, V. (2014). Sistema de diagnóstico OBD II. Fuente recuperado de <http://www.electronicar.net>.
- Villamar Aguirre, I. D. (2008). Estudio y Análisis de los Sistemas de Diagnóstico en los automóviles modernos, Sistemas OBD (Bachelor's thesis, Universidad del Azuay).
- Lopes, J. C. O. (2014). Diseño de un escáner automotriz OBDII Multi-protocolo. Universidad de San Carlos de Guatemala.
- Madariaga Revuelta, A. (2022). Domotización de un puesto de trabajo usando el microcontrolador ESP32.

Barrios Portales, D. (2022). Sistema para la adquisición, guardado e interpretación de datos provenientes de un vehículo.

Valverde González, V. L., Vaca Barahona, B. E., & Hidalgo Solórzano, G. X. (2025). Desarrollo web con Python y Flask.

Morales-Chan, M. A. (2023). Explorando el potencial de ChatGPT: Una clasificación de Prompts efectivos para la enseñanza.

Collaguazo, M. L. R. Q., Venegas, M. M. S. P., Guerrero, A. A. A., Freire, M. N. M., & Beltrán, M. S. H. C. (2022). Desarrollo híbrido con flutter. Ciencia Latina Revista Científica Multidisciplinar, 6(4), 4594-4609.

Hernández, N. L., & Fuentes, A. S. F. (2014). Computación en la nube. Mundo Fesc, 4(8), 46-51.

Galo Fariño, R. (2011). Modelo Espiral de un proyecto de desarrollo de software. Obtenido de <http://www.ojovisual.net/galofarino/modeloespiral.pdf>.

13. Anexos

- Implementación ESP32 OBD-II: <https://github.com/JuanQuizhpi/Implemetaci-nOBD-II-ESP32.git>
- Frontend - App de Flutter
<https://github.com/FAndresG02/Frontend-Flutter.git>
- Backend - Servidor de Flask
<https://github.com/FAndresG02/Backend-Flask.git>